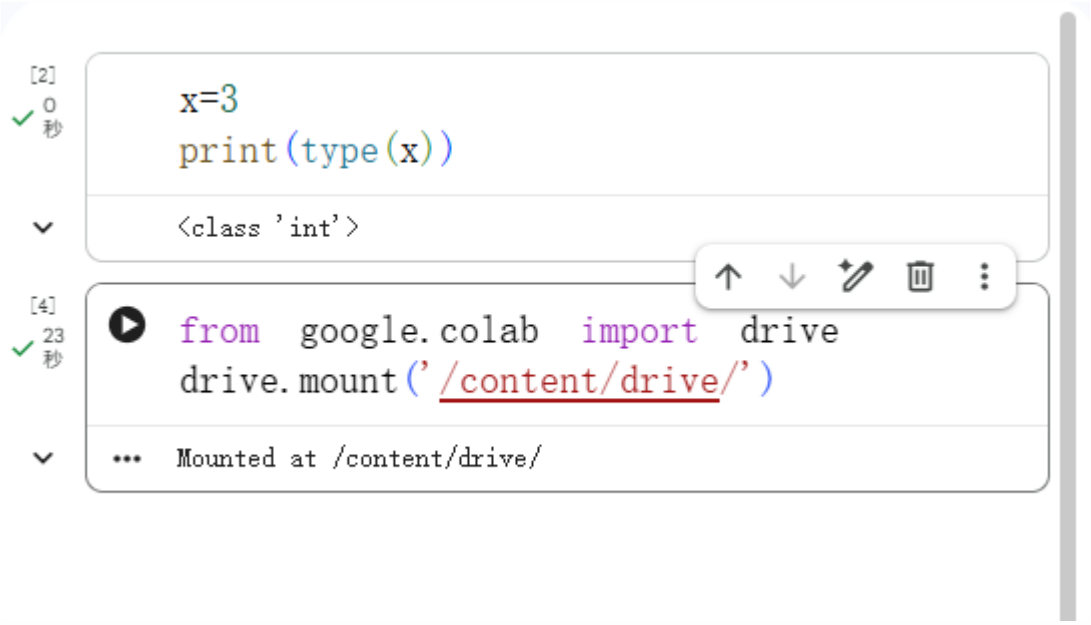
姓名：程鹏炜 学号：24130061003

姓名和学号?	**程鹏炜，24130061003**
本实验属于哪门课程?	中国海洋大学25秋《软件工程原理与实践》
实验名称?	实验2：深度学习基础
博客链接：	https://blog.csdn.net/cpw123_/article/details/167387481?sharetype=blogdetail&sharerId=167387481&sharerefer=PC&sharesource=cpw123_&spm=1011.2480.3001.8118

一、实验内容

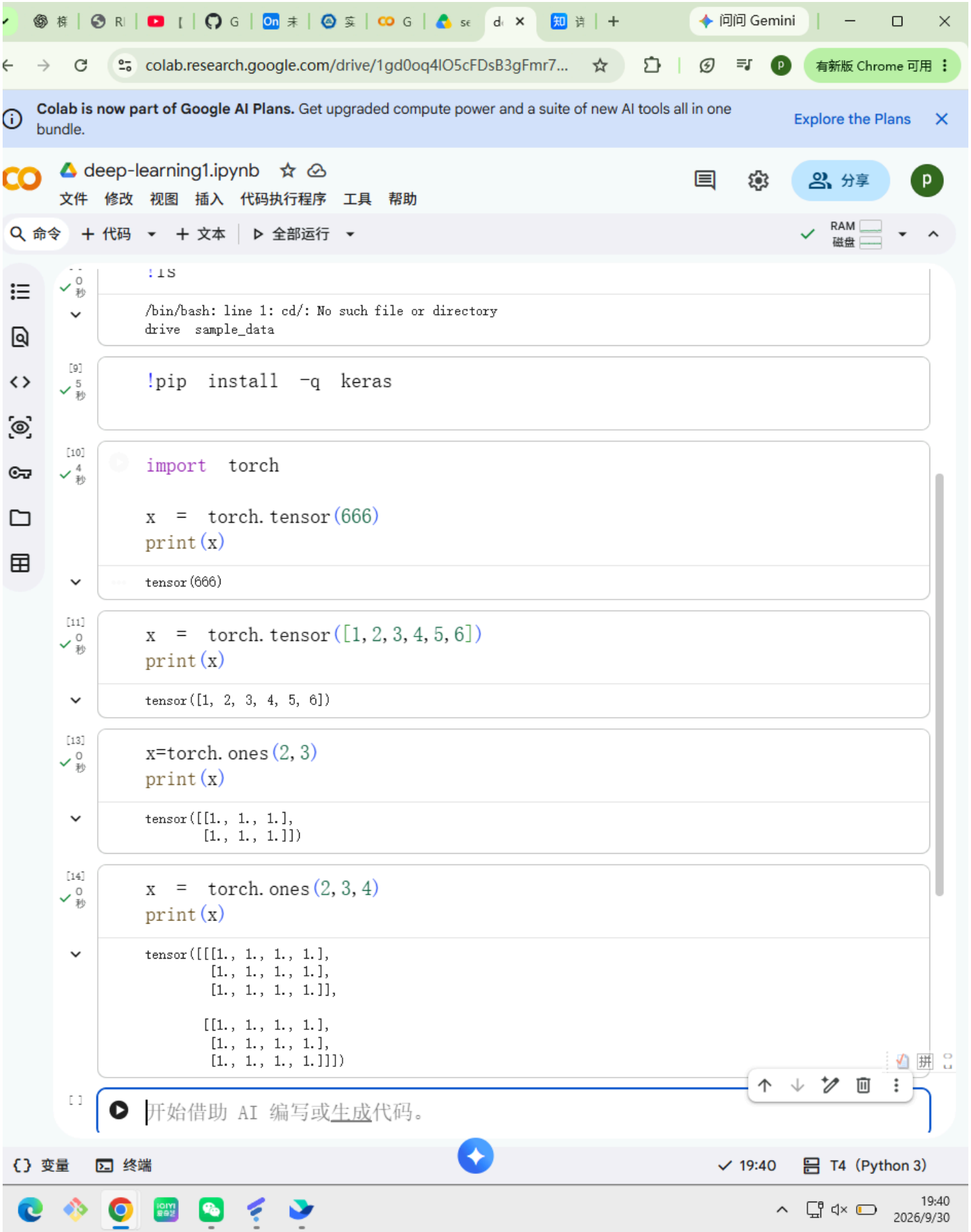
代码练习

1.colab完成基础配置：



挂在云盘方便创建文件夹管理文件。

2.基础pytorch训练----tensor语法：


问问 Gemini
colab.research.google.com/drive/1gd0oq4lO5cFDsB3gFmr7...
有新版 Chrome 可用
Colab is now part of Google AI Plans. Get upgraded compute power and a suite of new AI tools all in one bundle.
Explore the Plans
deep-learning1.ipynb
文件 修改 视图 插入 代码执行程序 工具 帮助
分享
命令 + 代码 + 文本 全部运行
RAM
磁盘
!ls
/bin/bash: line 1: cd/: No such file or directory
drive sample_data
!pip install -q keras
import torch
x = torch.tensor(666)
print(x)
tensor(666)
x = torch.tensor([1, 2, 3, 4, 5, 6])
print(x)
tensor([1, 2, 3, 4, 5, 6])
x=torch.ones(2, 3)
print(x)
tensor([[1., 1., 1.],
[1., 1., 1.]])
x = torch.ones(2, 3, 4)
print(x)
tensor([[[1., 1., 1., 1.],
[1., 1., 1., 1.],
[1., 1., 1., 1.]],
[[1., 1., 1., 1.],
[1., 1., 1., 1.],
[1., 1., 1., 1.]]])
开始借助 AI 编写或生成代码。
变量 终端
19:40
T4 (Python 3)
19:40
2026/9/30

处理基础数组

[15] 0秒

```
x=torch.rand(5, 3)
print(x)
```

```
tensor([[0.3009, 0.0379, 0.1751],
        [0.7199, 0.6916, 0.7405],
        [0.0183, 0.3779, 0.7581],
        [0.1178, 0.5059, 0.0632],
        [0.8917, 0.0741, 0.0725]])
```

[17] 0秒

```
x = torch.zeros(5, 3, dtype=torch.long)
print(x)
```

```
tensor([[0, 0, 0],
        [0, 0, 0],
        [0, 0, 0],
        [0, 0, 0],
        [0, 0, 0]])
```

[18] 0秒

```
z = torch.randn_like(x, dtype=torch.float)
print(z)
```

```
tensor([[-0.6298,  1.0187,  0.1836],
        [-0.0283,  0.3352,  0.5600],
        [-2.3132,  0.8662, -1.2078],
        [ 1.5963,  0.2565, -0.3968],
        [ 0.3670,  0.2887, -1.0409]])
```

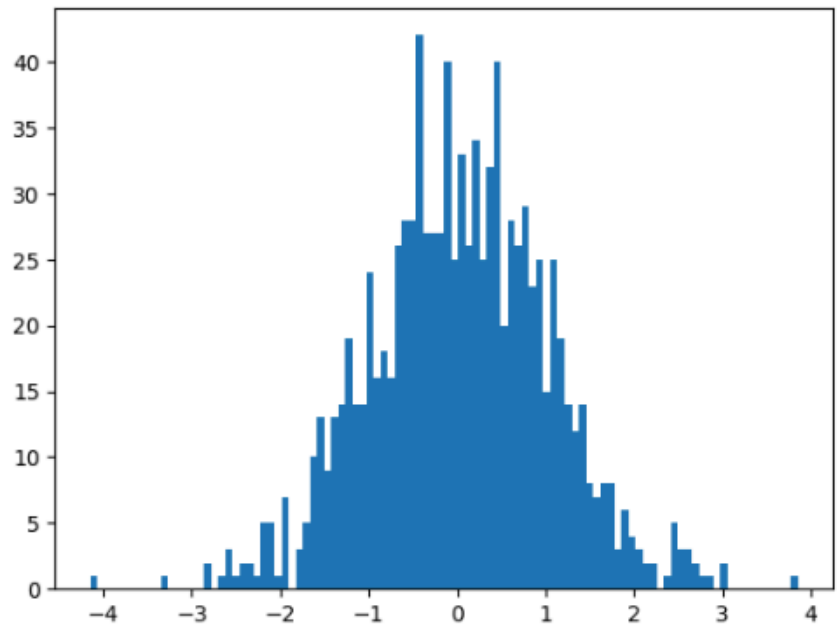
[ ]

开始借助 AI 编写或生成代码。

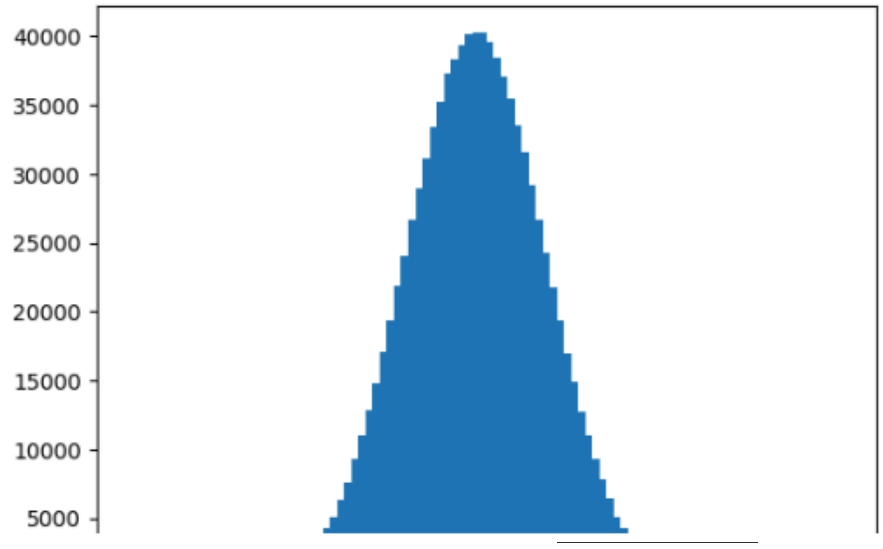
数据结构的

```
from matplotlib import pyplot as plt
import torch

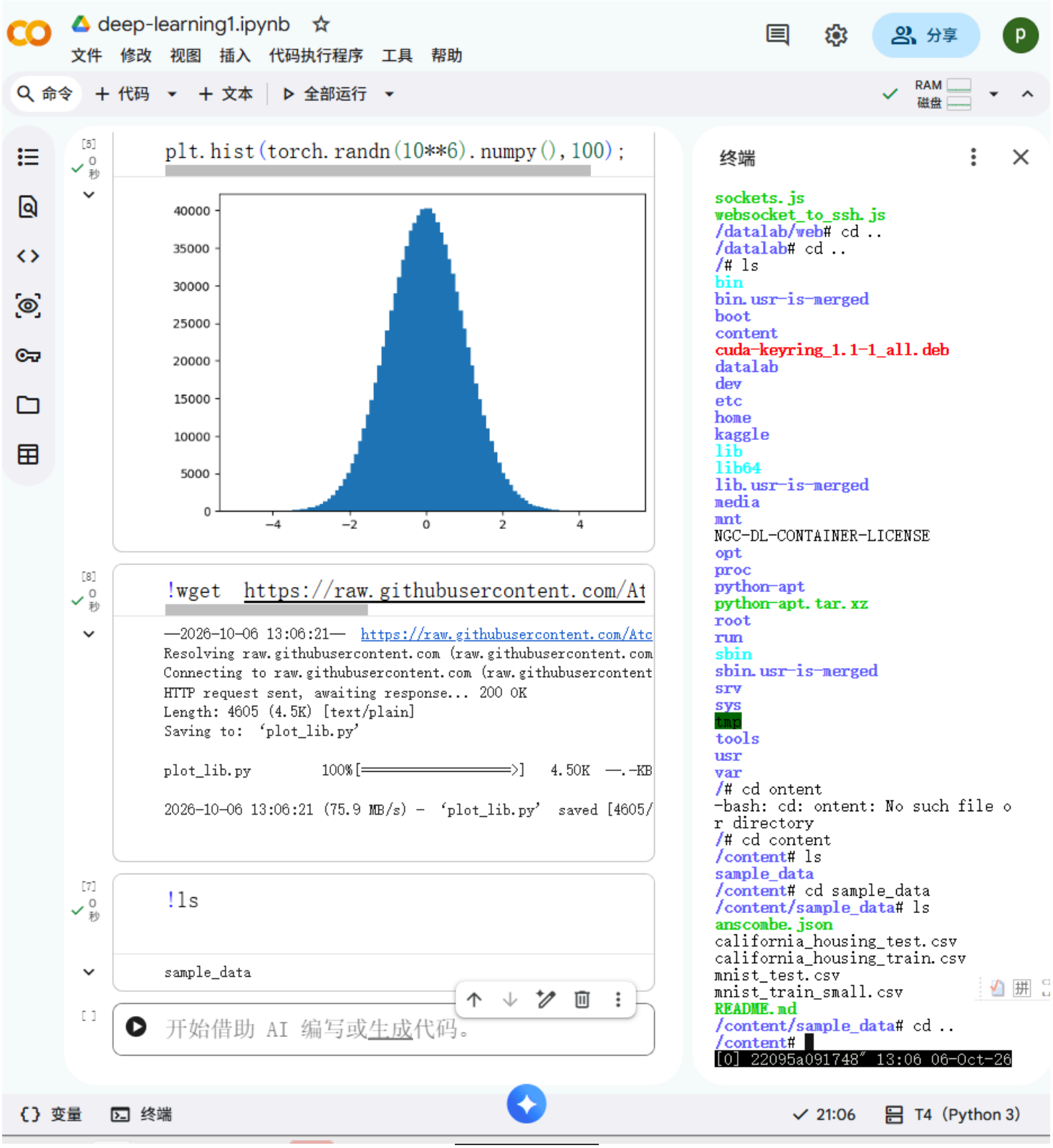
plt.hist(torch.randn(1000).numpy(), 100);
```




```
plt.hist(torch.randn(10**6).numpy(),100);
```



正态分布



优化正态分布和下载对应功能函数文件。

3.螺旋数据分类实践：

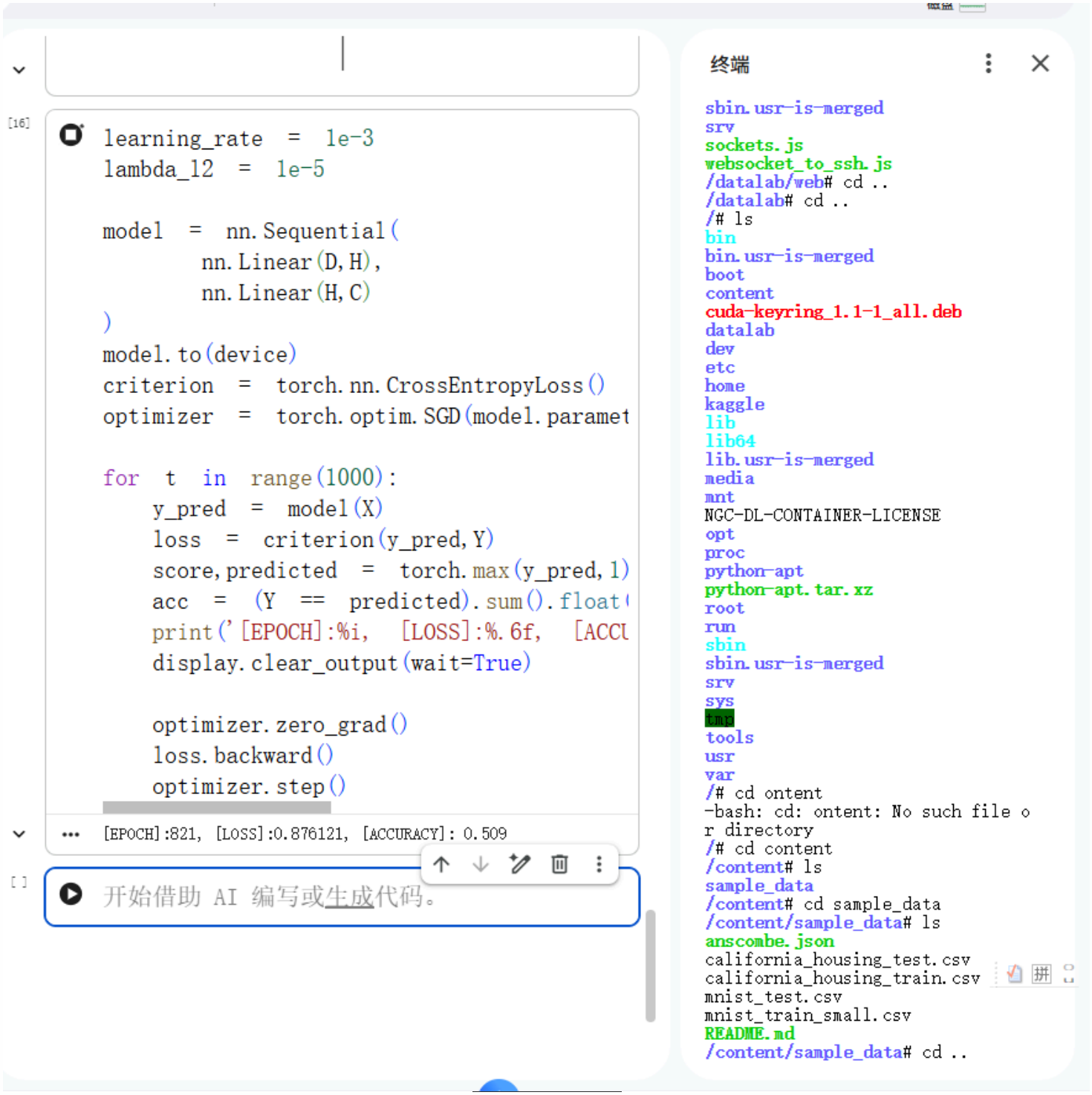
```
X = torch.zeros(N*C,D).to(device)
Y = torch.zeros(N*C,dtype=torch.long).to(
for c in range(C):
    index = 0
    t = torch.linspace(0,1,N)
    inner_var = torch.linspace((2*math.pi

    for ix in range(N*c,N*(c+1)):
        X[ix] = t[index]*torch.FloatTensc
        Y[ix] = c
        index +=
        print("Shap
        print("X:",X.size())
        print("Y:",Y.size())
```

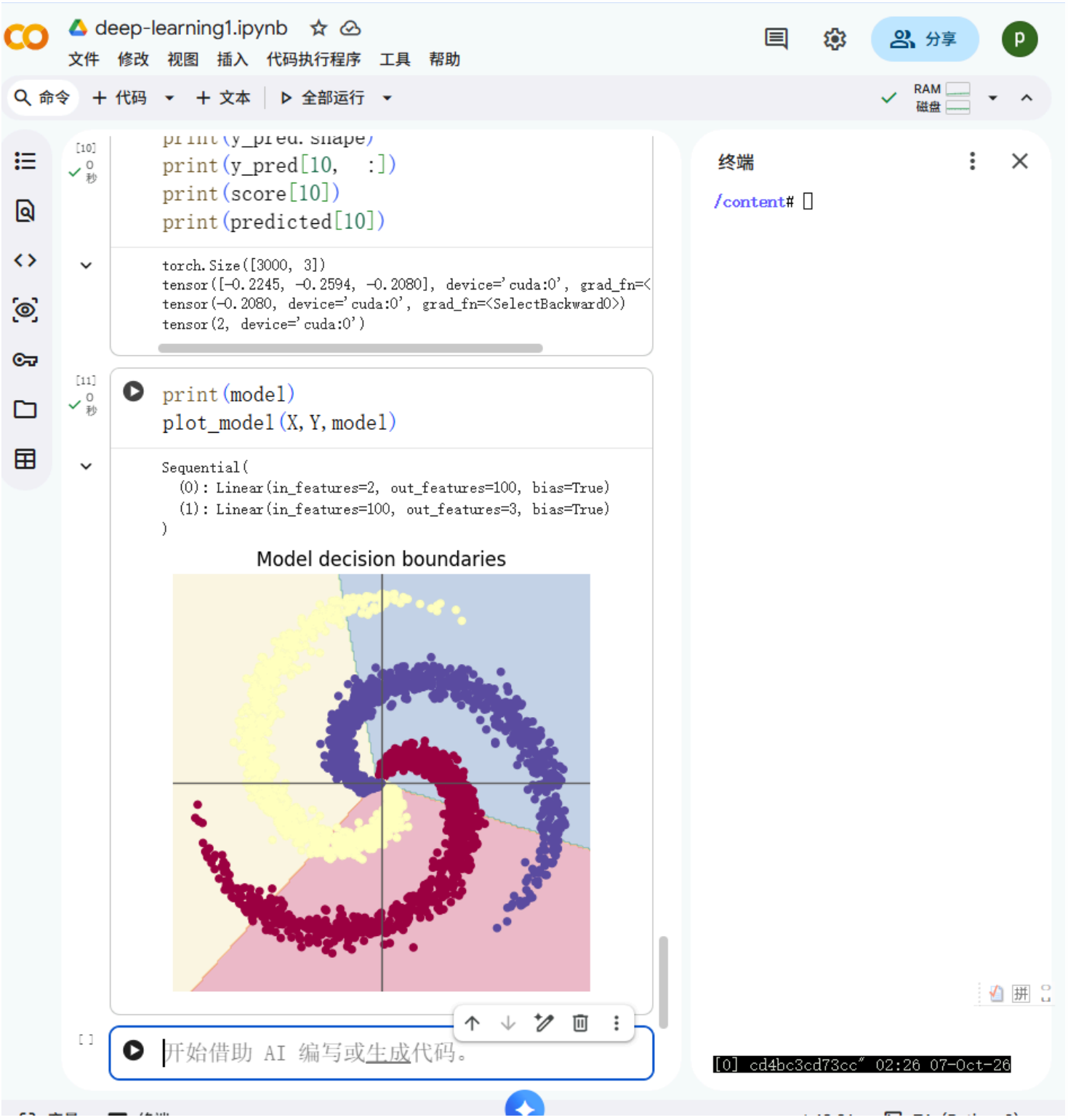
```
X.size
<built-in method size
```

```
流式输出内容被截断，只能显示最后 5000 行内容。
```

数据预处理简单处理维度通道channel



跑训练使用sgd优化器然后简单的线性linear层两层跑出来准确率很低，可视化结果：



然后改成relu的中间加入一层relu发现准确率几乎没这么上升，于是发现源代码里面是修改了优化器成为adam，而写的代码还在使用sgd所以没有进一步很明显的提升可以看到没有使用较好的优化器的时候根本训不起来，loss下降和relu的拟合都很差，可能在saddle/loacl point一直打转没法出山谷。原始实验链接nyu的可以参考这个https://oucaigroup.feishu.cn/wiki/Nbddwe54fiolLFkoslfcdYcAnZe

然后发现之前有照片位置不对可能也会降低一定的准确率：

于是进行修正

```
!wget -O ziegler.png https://raw.githubu
```

```
--2026-10-07 02:35:12--  https://raw.githubusercontent.com/Atc
Resolving raw.githubusercontent.com (raw.githubusercontent.com
Connecting to raw.githubusercontent.com (raw.githubusercontent
HTTP request sent, awaiting response... 200 OK
Length: 361037 (353K) [image/png]
Saving to: 'ziegler.png'

ziegler.png         100%[===================>] 352.58K  --.-KB

2026-10-07 02:35:12 (13.0 MB/s) - 'ziegler.png' saved [36103
```

```
from IPython.display import Image, disp

display(Image("ziegler.png"))
```



然后换成adam准确率到了九十多

```
cate  =  1e-3
  =  1e-5


 nn.Sequential(
1.Linear(D,H),
1.ReLU(),
1.Linear(H,C)

device)
  =  torch.nn.CrossEntropyLoss()
  =  torch.optim.Adam(model.parameters(),lr

1  range(1000):
d  =  model(X)
 =  criterion(y_pred,Y)
 predicted  =  torch.max(y_pred,1)
=  (Y  ==  predicted).sum().float()/len(Y)
('[EPOCH]:%i,  [LOSS]:%.6f,  [ACCURACY]:  %
ay.clear_output(wait=True)

izer.zero_grad()
backward()
izer.step()
```

```
[EPOCH]:999, [LOSS]:0.193811, [ACCURACY]: 0.934
```

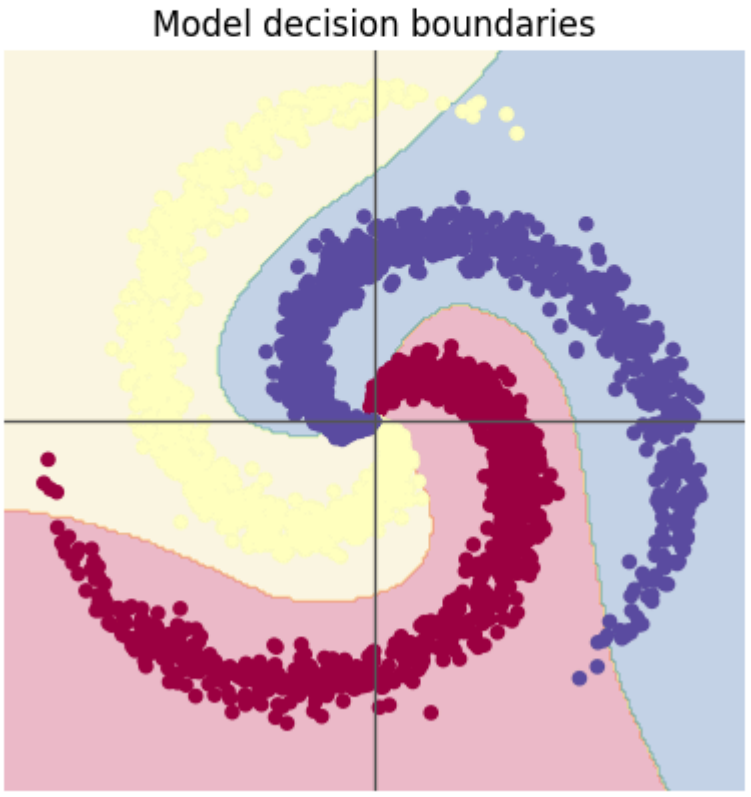
```
plot_model(X,Y,model)
print(model)
```

```
Sequential(
  (0): Linear(in_features=2, out_features=100, bias=True)
```

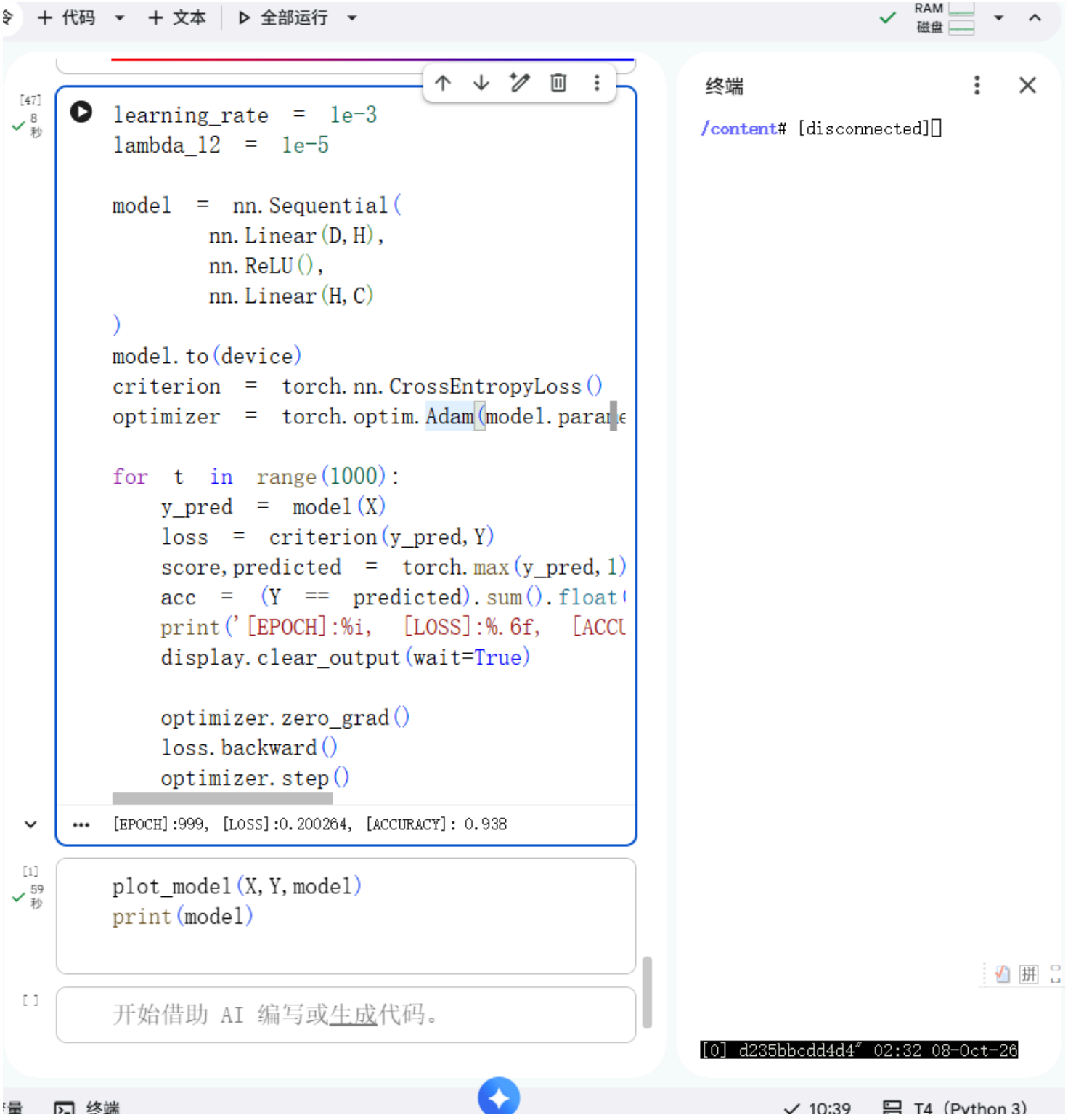
/content#

```
plot_model(X, Y, model)
print(model)
```

```
Sequential(
  (0): Linear(in_features=2, out_features=100, bias=True)
  (1): ReLU()
  (2): Linear(in_features=100, out_features=3, bias=True)
)
```



可视化，然后再换成adamw试试，并且去掉relu只单独使用adam，对比发现效果也不是很好


+ 代码
+ 文本
全部运行
RAM
磁盘
[47]
8
秒
learning_rate = 1e-3
lambda_l2 = 1e-5
model = nn.Sequential(
nn.Linear(D,H),
nn.ReLU(),
nn.Linear(H,C)
)
model.to(device)
criterion = torch.nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.para
for t in range(1000):
y_pred = model(X)
loss = criterion(y_pred,Y)
score,predicted = torch.max(y_pred,1)
acc = (Y == predicted).sum().float(
print('[EPOCH]:%i, [LOSS]:%.6f, [ACCU
display.clear_output(wait=True)
optimizer.zero_grad()
loss.backward()
optimizer.step()
[EPOCH]:999, [LOSS]:0.200264, [ACCURACY]: 0.938
[1]
59
秒
plot_model(X,Y,model)
print(model)
[ ]
开始借助 AI 编写或生成代码。
终端
/content# [disconnected][]
[0] d235bbcdd4d4" 02:32 08-Oct-26
终端
10:39
T4 (Python 3)

```
learning_rate  =  1e-3
lambda_12  =  1e-5

model  =  nn.Sequential(
        nn.Linear(D,H),
        nn.ReLU(),
        nn.Linear(H,C)
)
model.to(device)
criterion  =  torch.nn.CrossEntropyLoss()
optimizer  =  torch.optim.AdamW(model.paran

for  t  in  range(1000):
    y_pred  =  model(X)
    loss  =  criterion(y_pred,Y)
    score,predicted  =  torch.max(y_pred,1)
    acc  =  (Y  ==  predicted).sum().float(
    print('[EPOCH]:%i,  [LOSS]:%.6f,  [ACCU
    display.clear_output(wait=True)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

```
[EPOCH]:999, [LOSS]:0.180543, [ACCURACY]: 0.948
```

单纯adam只有50，所以激活函数和优化器同样重要缺一不可

```
learning_rate  =  1e-3
lambda_12  =  1e-5

model  =  nn.Sequential(
        nn.Linear(D,H),

        nn.Linear(H,C)
)
model.to(device)
criterion  =  torch.nn.CrossEntropyLoss()
optimizer  =  torch.optim.Adam(model.parame

for  t  in  range(1000)
    y_pred  =  model(X)
    loss  =  criterion(y_pred,Y)
    score,predicted  =  torch.max(y_pred,1)
    acc  =  (Y  ==  predicted).sum().float(
    print('[EPOCH]:%i,  [LOSS]:%.6f,  [ACCU
    display.clear_output(wait=True)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

Tensor: y_pred

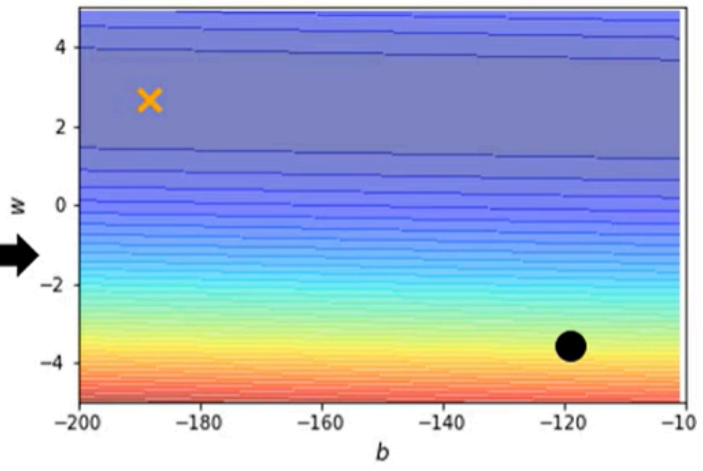
Tensor with shape

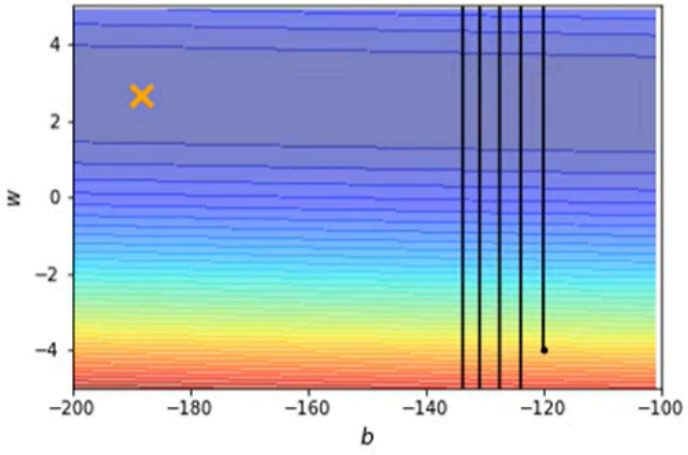
```
[EPOCH]:999, [LOSS]:0.775228, [ACCURACY]: 0.507
```

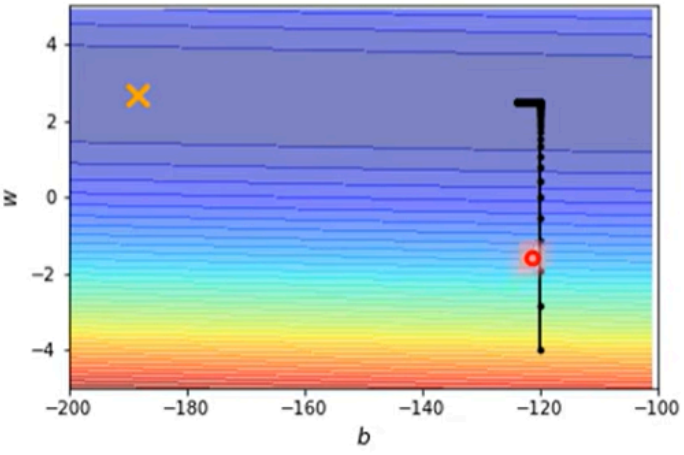
4.思考总结：在发展中的深度学习中最让人头疼的事情就是训练训不起来，（目前已经比较成熟了有很多的万能优化器比较万精油的算法和function）关于relu的优点不用多说超越sigmoid的激活函数，让非线性引入，虽然看到他的函数图像只比正比例函数多一个平台但是这样两个relu拼接出来一个sigmoid然后无数个relu能拟合所有的复杂的函数，从而将这个非线性引入。

Training can be difficult even without critical points.

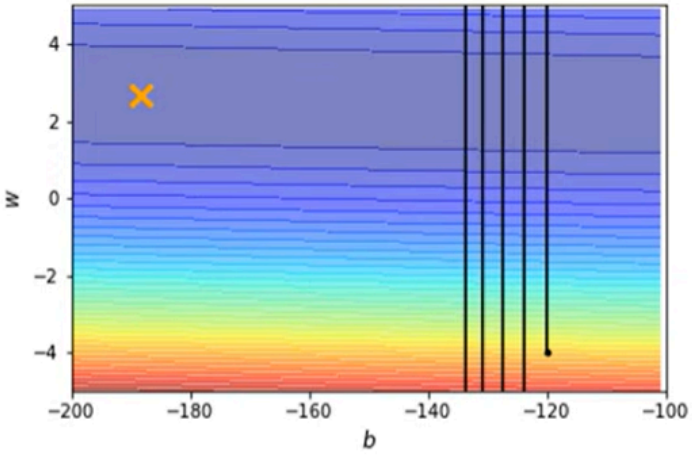
This error surface is convex.

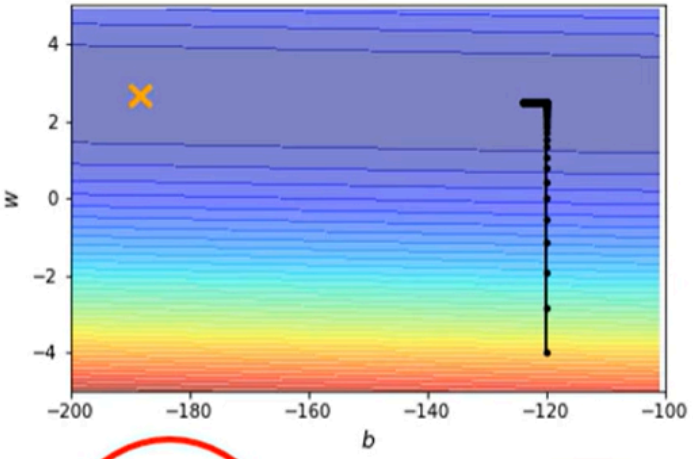






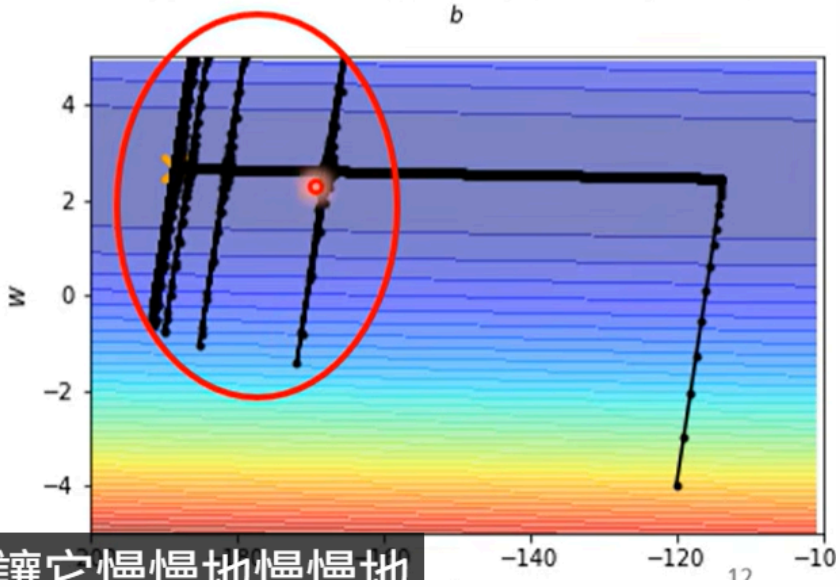
Without Adaptive Learning Rate



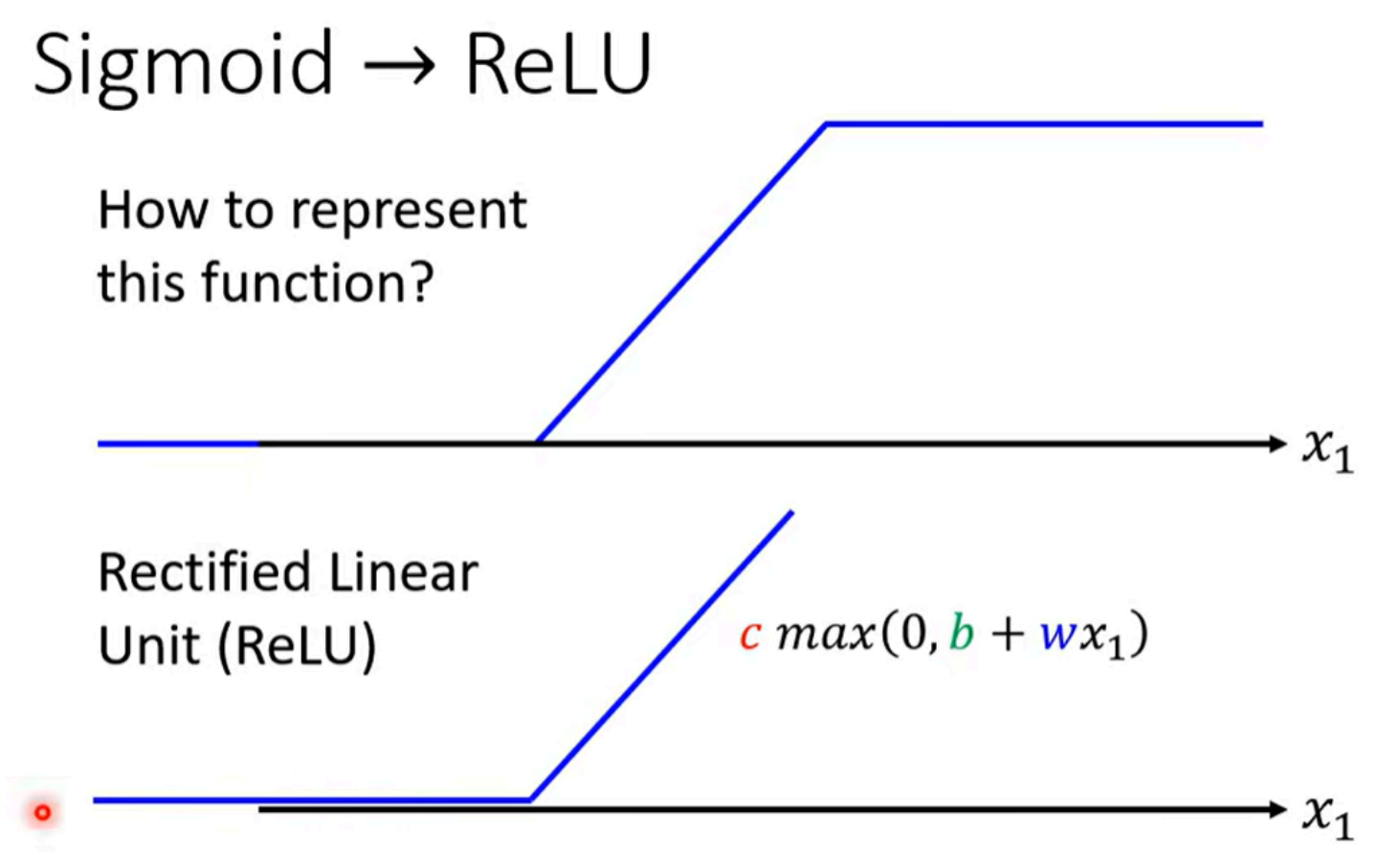


$$\theta_i^{t+1} \leftarrow \theta_i^t - \frac{\eta}{\sigma_i^t} g_i^t$$

$$\sigma_i^t = \sqrt{\frac{1}{t+1}\sum_{i=0}^{t}\left(g_i^t\right)^2}$$



有磨擦力讓它慢慢地慢慢地

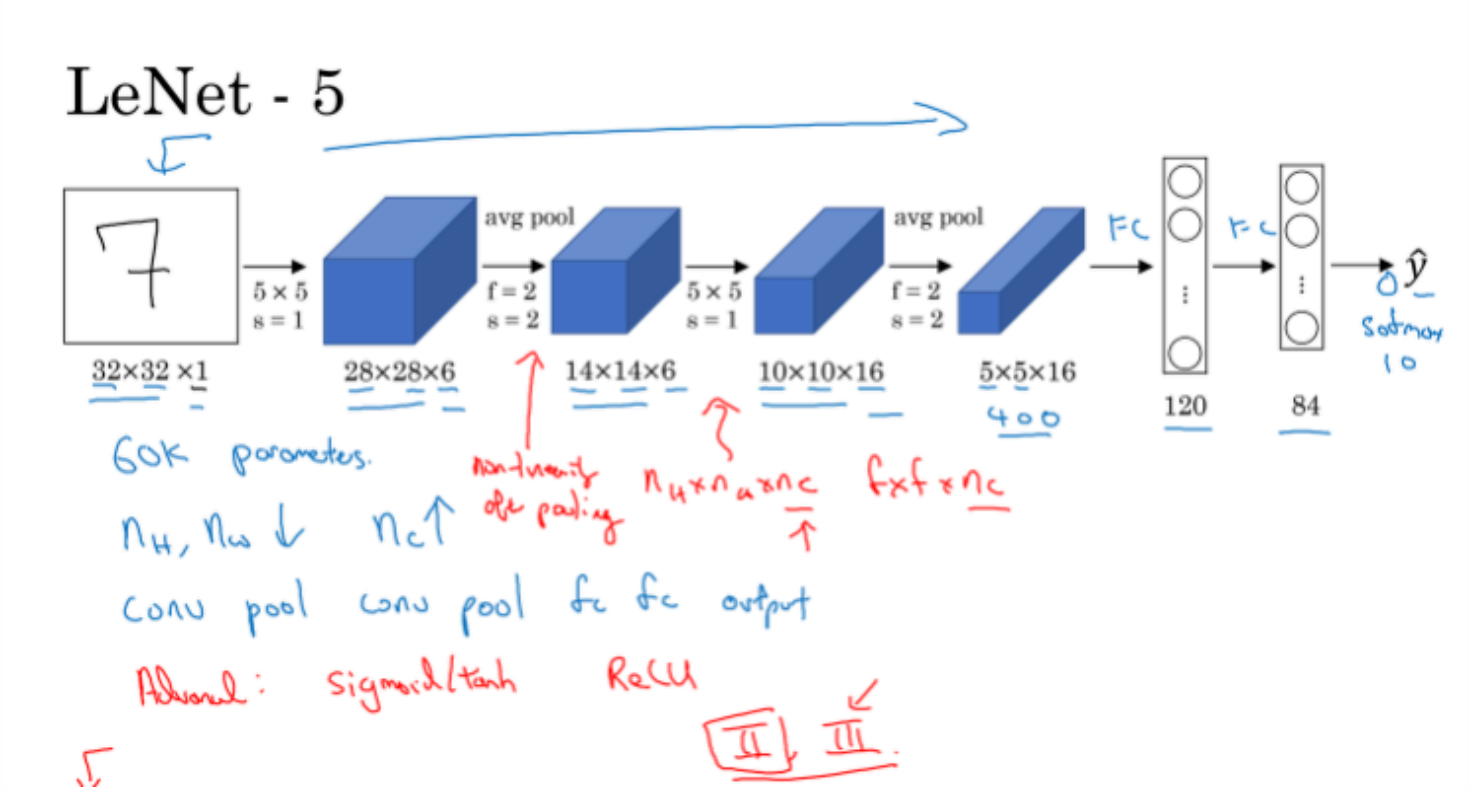


然后就是简单的螺旋数据问题里面没有引入评估机制，对应任何学科来说评估机制都非常重要，而我们有了评估才能便于确定训练什么时候停止以及最优model保存哪一个参数权重，所以明显可以看到训练过程中有准确率比最后一轮要高或者loss更低。

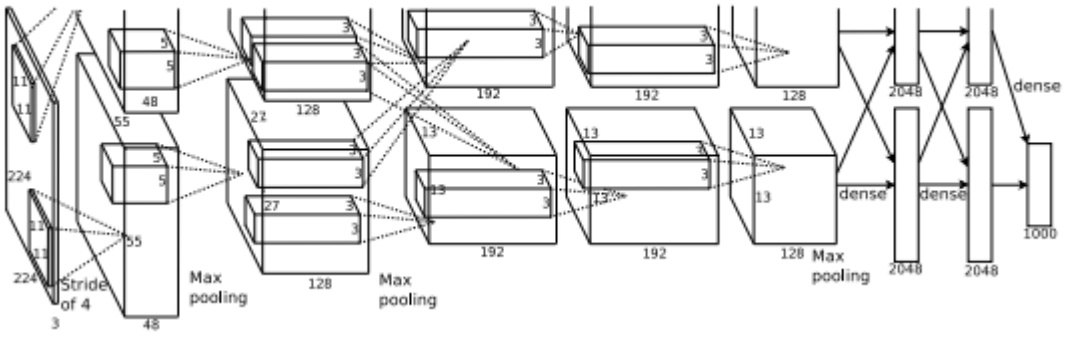
二、问题总结与体会

I.AlexNet有哪些特点？为什么可以比LeNet取得更好的性能？

1.简单看一下LeNet的网络架构



2.总共七层的cnn.其中pooling使用的是平均池化相当于在n*n的矩阵内选取平均值吧作为特征，而不是采用最大池化。参数较少，激活函数使用的是古老战神sigmoid和年轻一点点的战神 tanh。



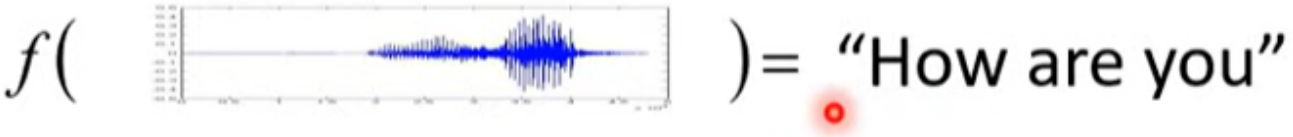
3.而这个alexnet也算是远古真神，第一次拉大参数量训练终于能train这么大的一个network了，首先max pooling采样保留的特征更加显著，然后激活函数使用了另一个真神--ReLU()这个效果激活函数确实好。首先根据BP算法，梯度反向传播的时候是要交叉相乘weight的其实就是激活函数的一阶导数，然而sigmoid也好tanh也好他们都是曲线并且梯度会存在很小的情况，那么反向传播如果模型非常的deep那么就出现了梯度消失的问题，而relu的导数恒为1（>0)的时候能规避这个问题，而且相对于指数函数和tan正切函数，线性函数倒数更好计算，也更容易加深deep network的层数让我们实现大参数量

4.其次relu具有稀疏激活的特性，大多数的神经元未激活就是0非常方便减少了大量计算，一定程度上能减缓overfitting

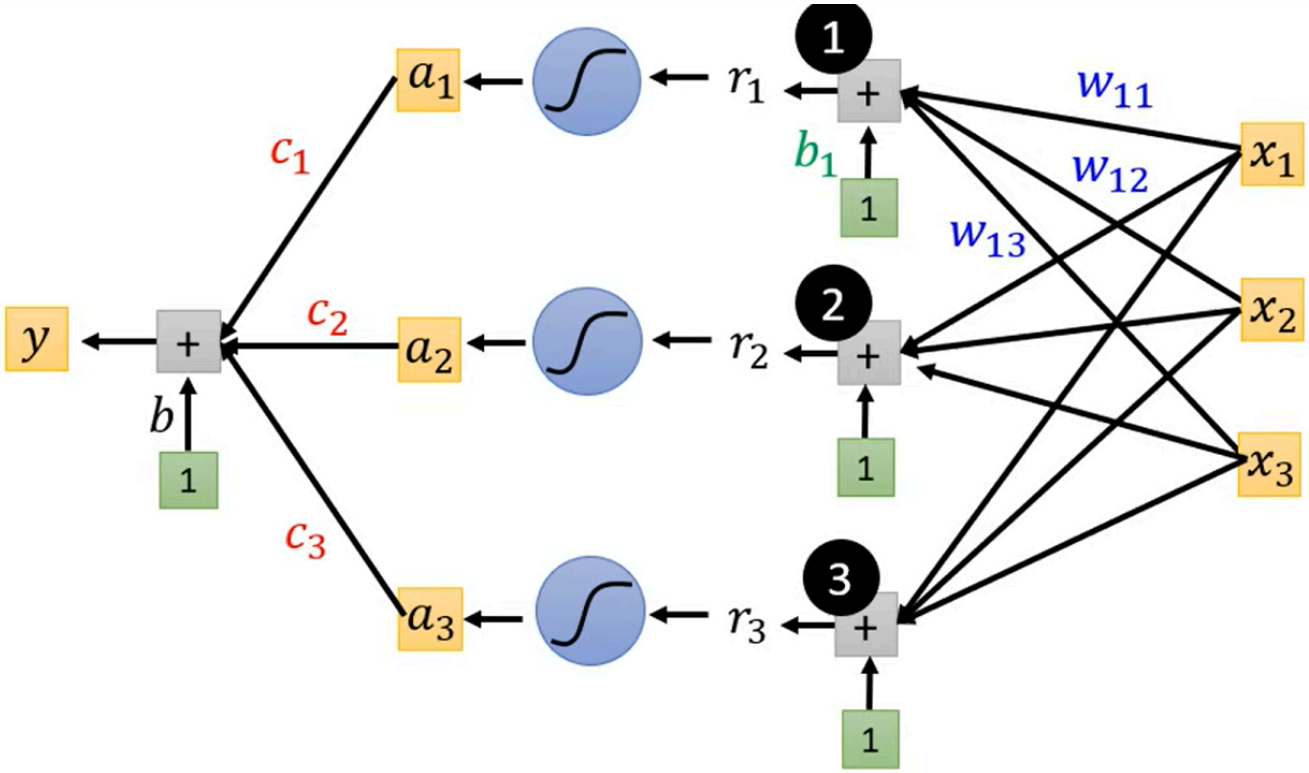
5.最后就是dropout了随机丢弃神经元，这个每次学习的相当于是个子网络，最大的好处就是增强鲁棒性，但是我不知道怎么证明他不会影响一写nueron之间的联系会不会被隐藏?

II.激活函数有哪些作用?

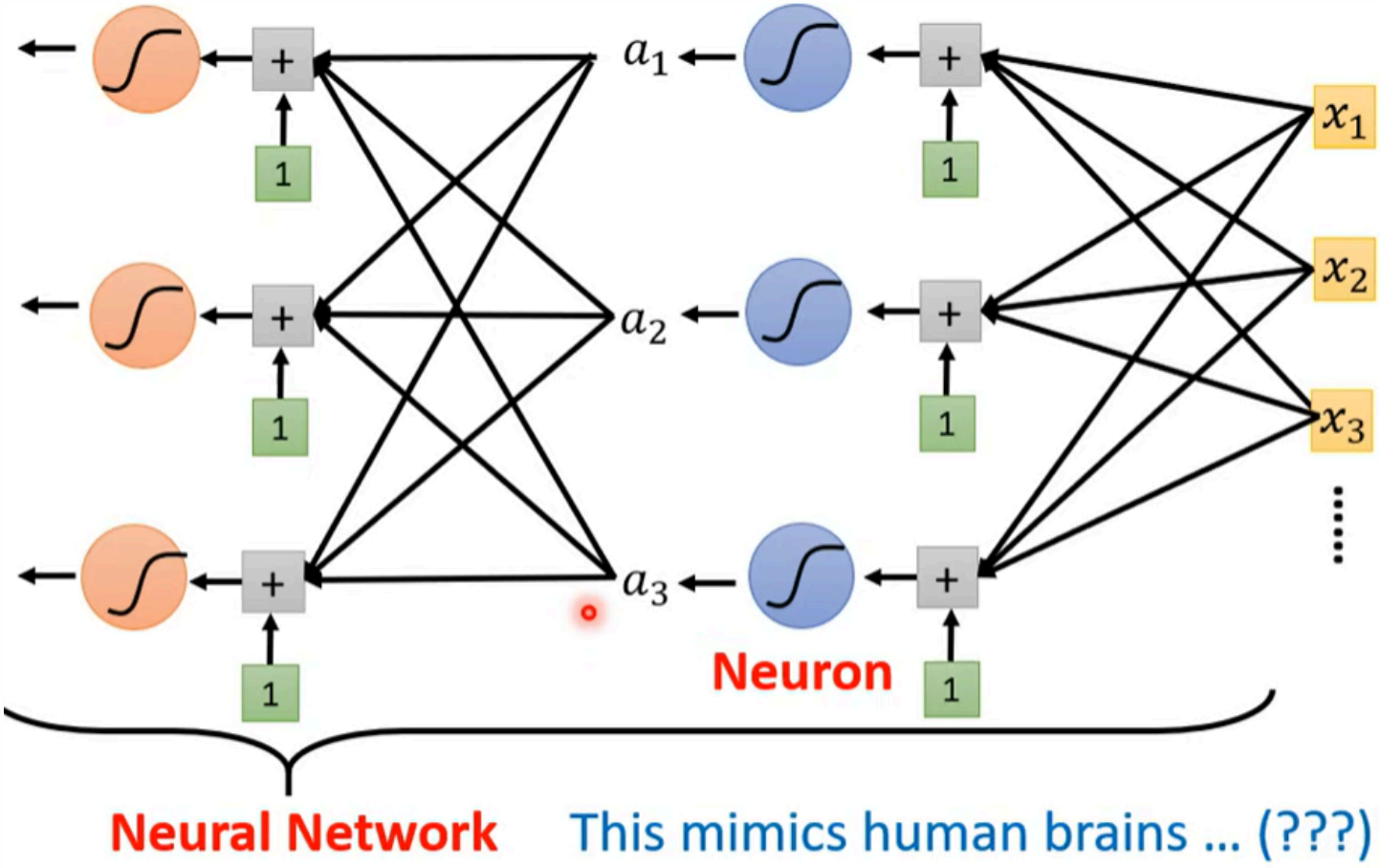
1.激活函数首先把输入的feature或者说变量通过函式转换成特定的值，你首先肯定要通过矩阵乘法把参数和权重加权求和，但是求和的出来的一定能拟合实际的内容吗？我们需要一个桥梁把他转换成为可以拟合真实曲线真实关系的function，这就是激活函数，激活函数选取以前sigmoid现在relu，他们能拟合出无限的function任何的函数关系都能被拟合，而我们训练的目标就是找到一个function来产生输出尽可能的接近我们的实际值，类似于下图。



2.我们分析一下，network的工作是什么：线性求和wi*xi+bi这样的公式，我就不写公式了太麻烦了放个计算图（本文图片大多截图自李宏毅的ML课程中，截的相关的内容做说明，直接自己画图不太现实）







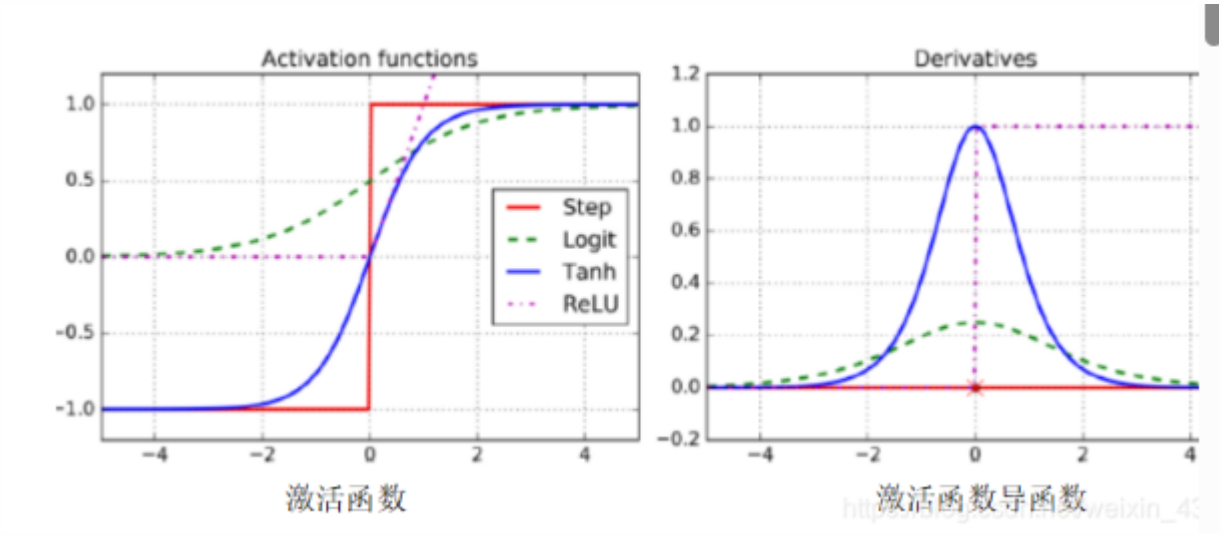
那时候是neural network 后来cnn来了tf来了就叫deep network了，引入卷积技术，但是这样neural nw中的一列神经元还有激活函数封装成了一个layer，激活函数还是起到了非线性化的重要作用

2.不同场景下面我们要选择不同的激活函数，分类可能你就会用softmax，虽然很多时候relu是个万精油，但是dead neuron也会导致很多问题。

3.说点空洞的，激活函数能增强网络的表示能力和学习能力。

III.梯度消失现象是什么？

1.这个在前文介绍relu的时候有提到过。梯度消失首先指的是反向传播过程中经过激活函数需要×激活函数的一阶导数导致如果导数小于1，梯度越来愈小，在比较深的network中最终消失趋近于0。



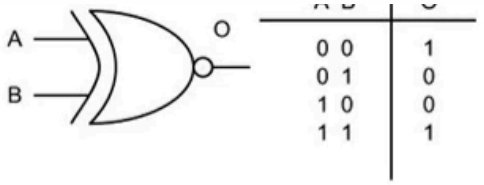
2.造成这个问题的主要原因就是激活函数选取的较为饱和的函数（比如sigmoid这种），他们的导函数值域可以说是R导致会出现一阶导数很小的情况，然而使用relu之后梯度恒传播不会经过激活函数的时候衰减。

3.残差也可以有效缓解，我们保留之前的部分只对增量部分传播，这样子永远不会消失了梯度。

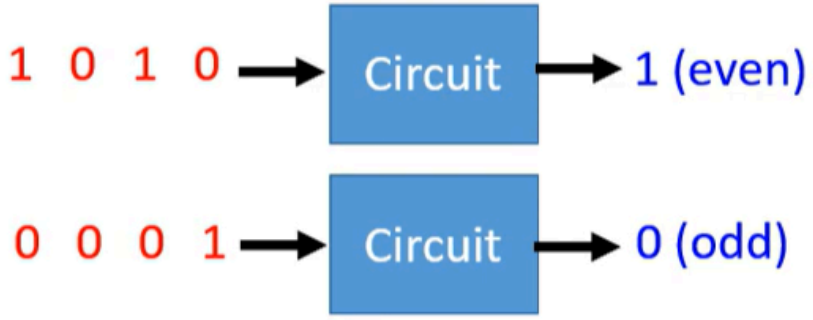
IV.神经网络是更宽好还是更深好?

1.首先肯定是deep比fat好。第一点就是deep的神经元k层可以制造出来2k次方个linear有这么多的函数去拟合实际，他是指数增长而我们的fat network只能产生k个，线性增长，那么显然同样的任务deep的需要的参数更少。原理证明就像逻辑门一样，并联的逻辑电路能产生2 k次方的真值表而不是串联。**

Analogy – Logic Circuits

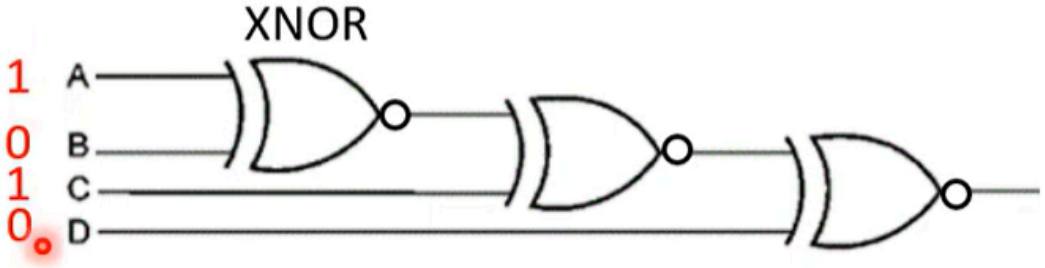


- E.g., ***parity check***



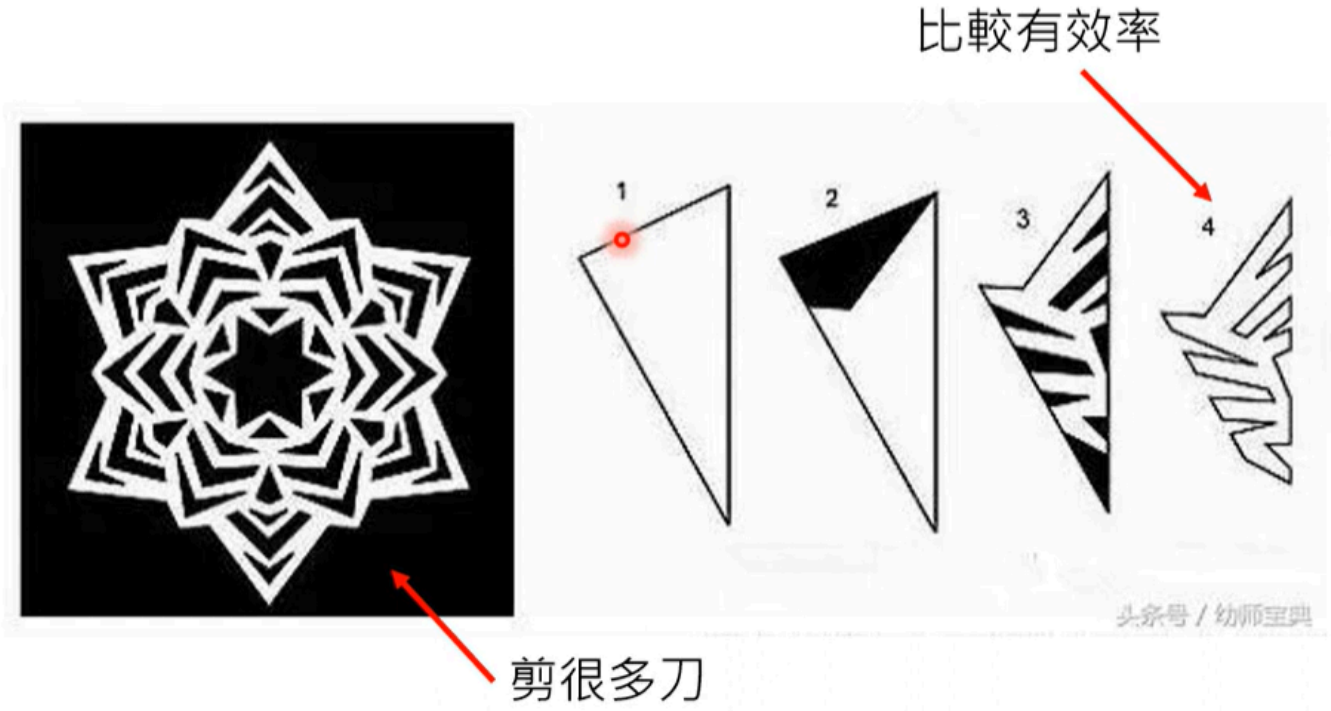
For input sequence with d bits,

Two-layer circuit need $O(2^d)$ gates.

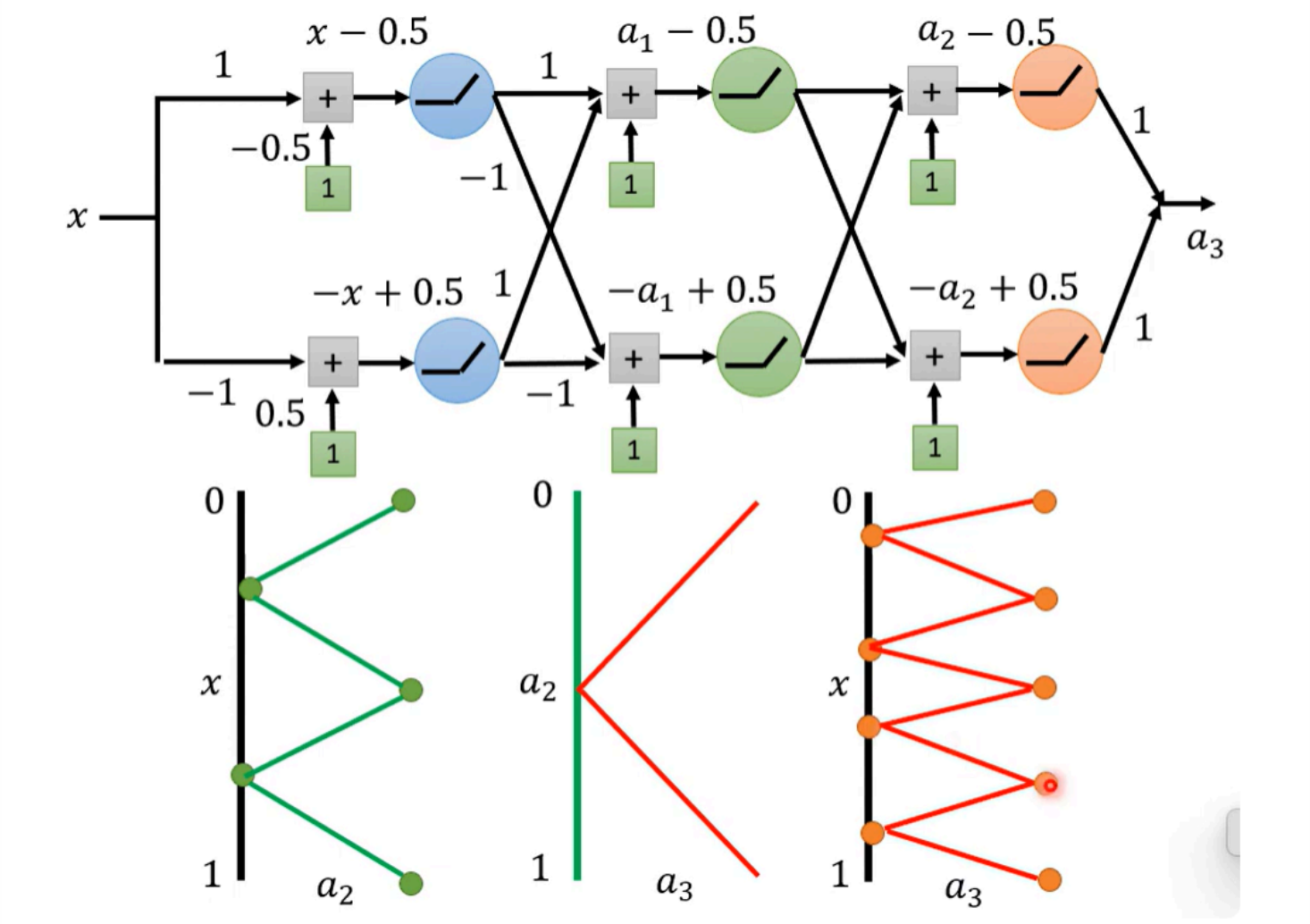


2.网络变得更有效果了更加聪明

More Analogy



兑成剪的准确率精准度一定更高，我们看下面这个图，从x映射到n layers的输出，看看输出是不是变得更加的多样，像信号一样它能够表征更多的信息



V.为什么要使用Softmax?

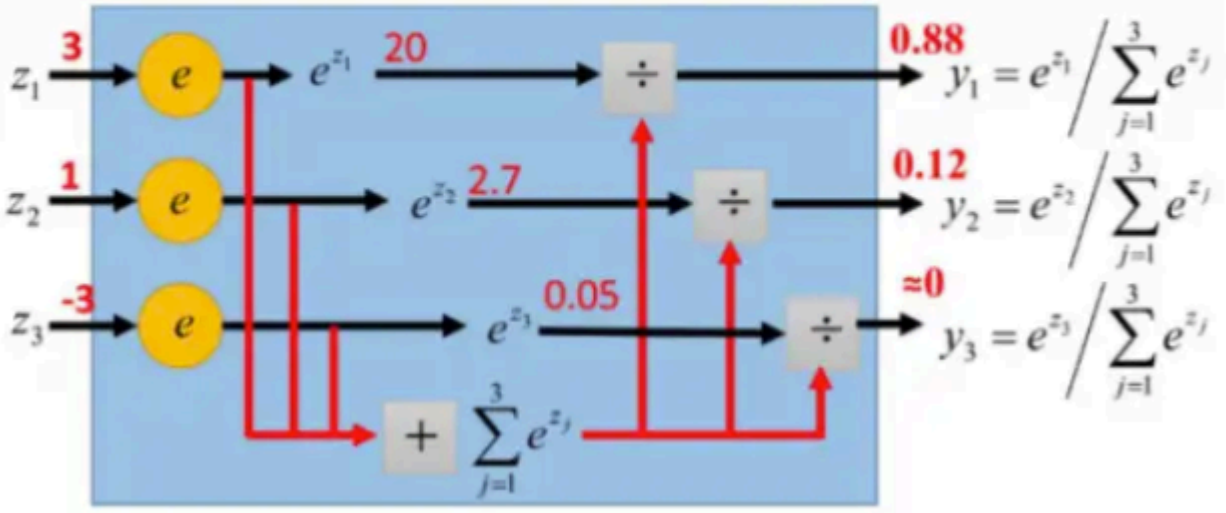
1.关于这个首先我们要知道softmax是个什么，softmax是一个归一函数，总和为1，一般作为一个全连接层的分类器使用，他能把你输入的x最终转成类似于权重比例的数值，一眼就能看出来谁分数最高，这样模型就能给出置信度和最终输出的classification是哪一个？就是概率分布产生器，而有时候也被作为激活函数使用。

- Softmax layer as the output layer

Probability:

- $1 > y_i > 0$
- $\sum_i y_i = 1$

Softmax Layer



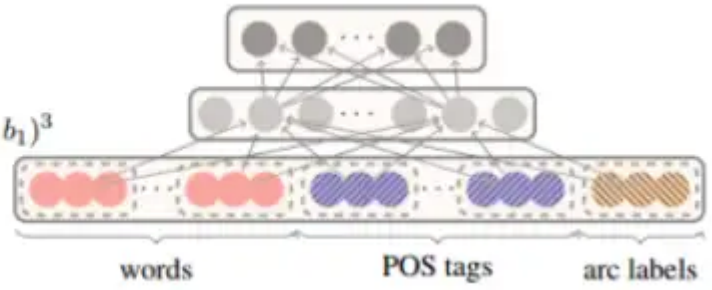
Softmax layer:

$p = \texttt{softmax}(W_2 h)$

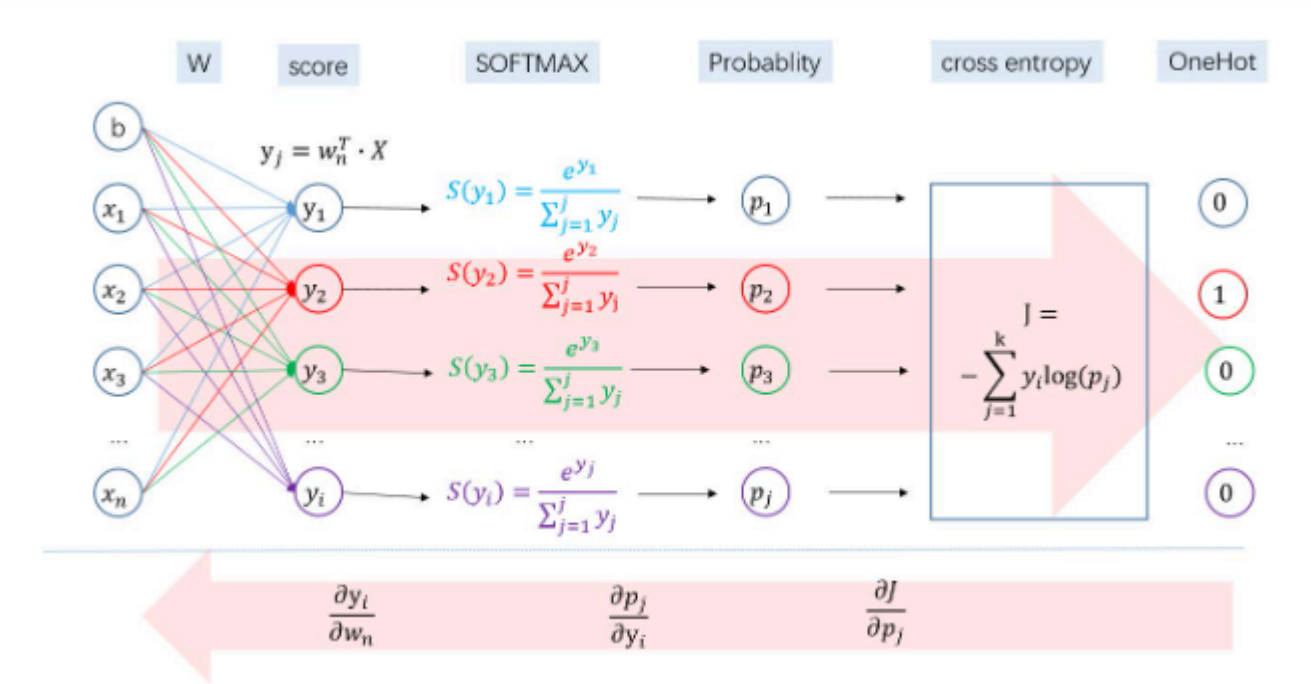
Hidden layer:

$h = (W_1^w x^w + W_1^t x^t + W_1^l x^l + b_1)^3$

Input layer: $[x^w, x^t, x^l]$



2.最重要的就是梯度求导反向传播的过程，softmax函数让这个过程大大简化。如果选出来其中一类直接在数值上-1然后再bp就可以了，当然要配合crossentropy使用，基本上他俩连体的。

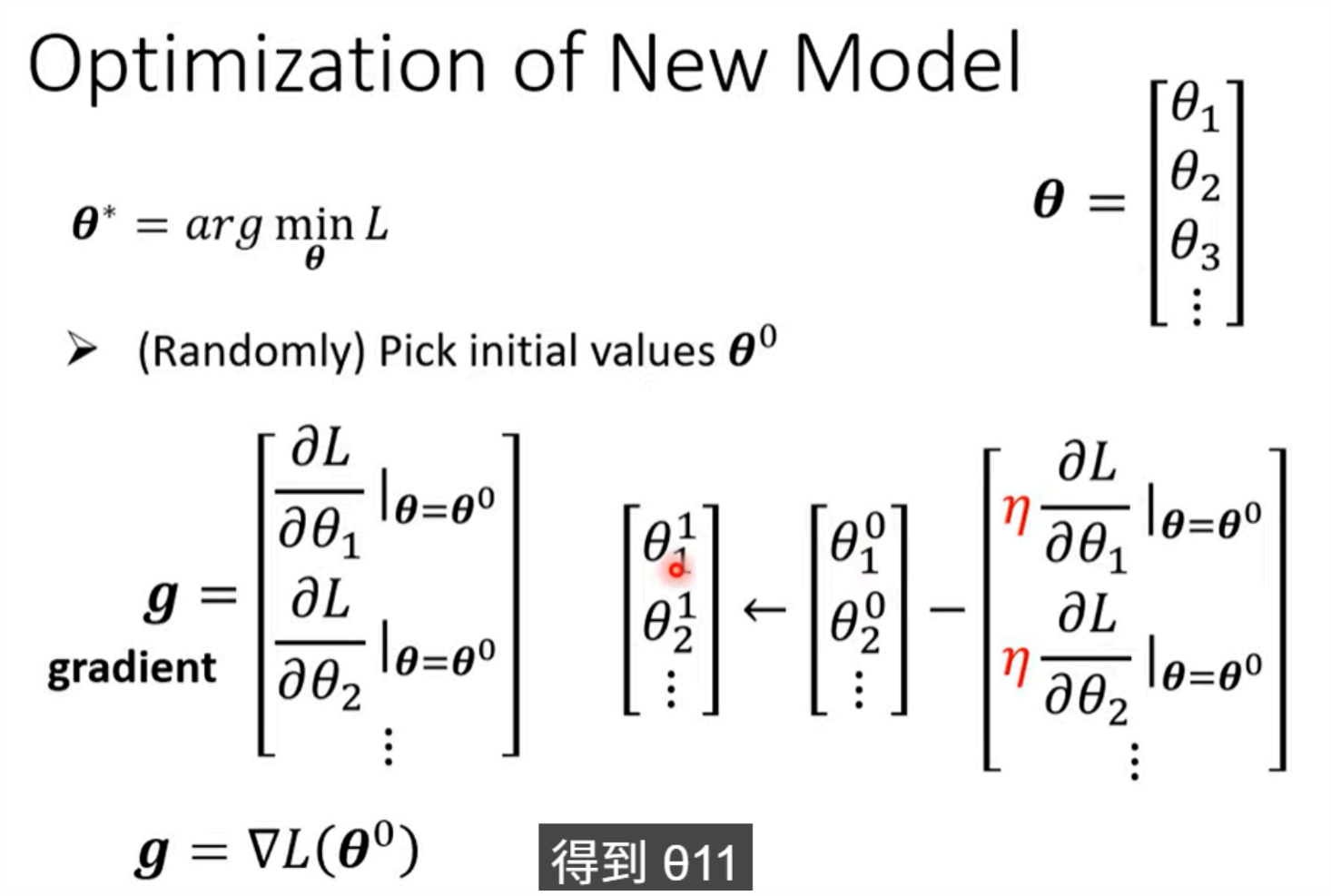


数学证明就不看了简单放一个完整bp过程，只需要直到最后推到出来反向传播的时候在对应class-1就得到了反向传播的导数，大大减少计算量而且本身也有非常出色的分类啊转化为非线性的概率分布的能力。

VI.SGD 和 Adam 哪个更有效?

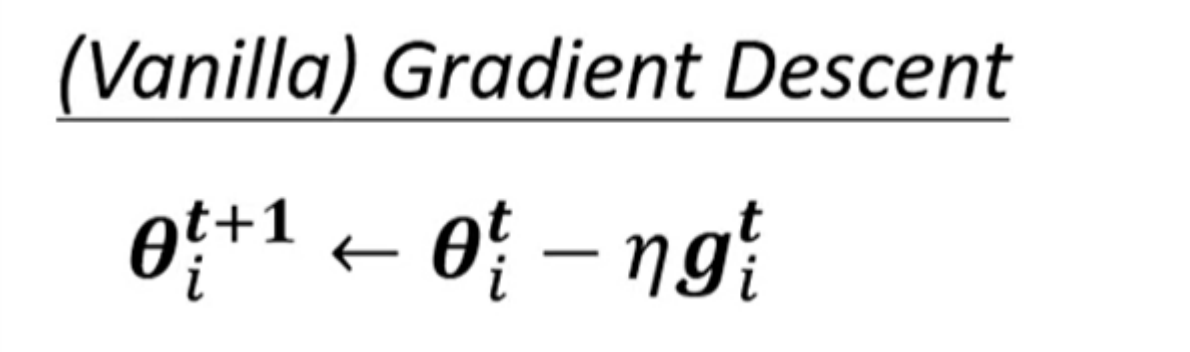
大多时候一定是Adam更好.首先介绍一下两者：

1.两者都是属于我们的optimizer的范畴都是优化器，为了能够快速的更新模型的参数，这个参数就是使得我们定义出来的loss取到最小值的时候的参数集合

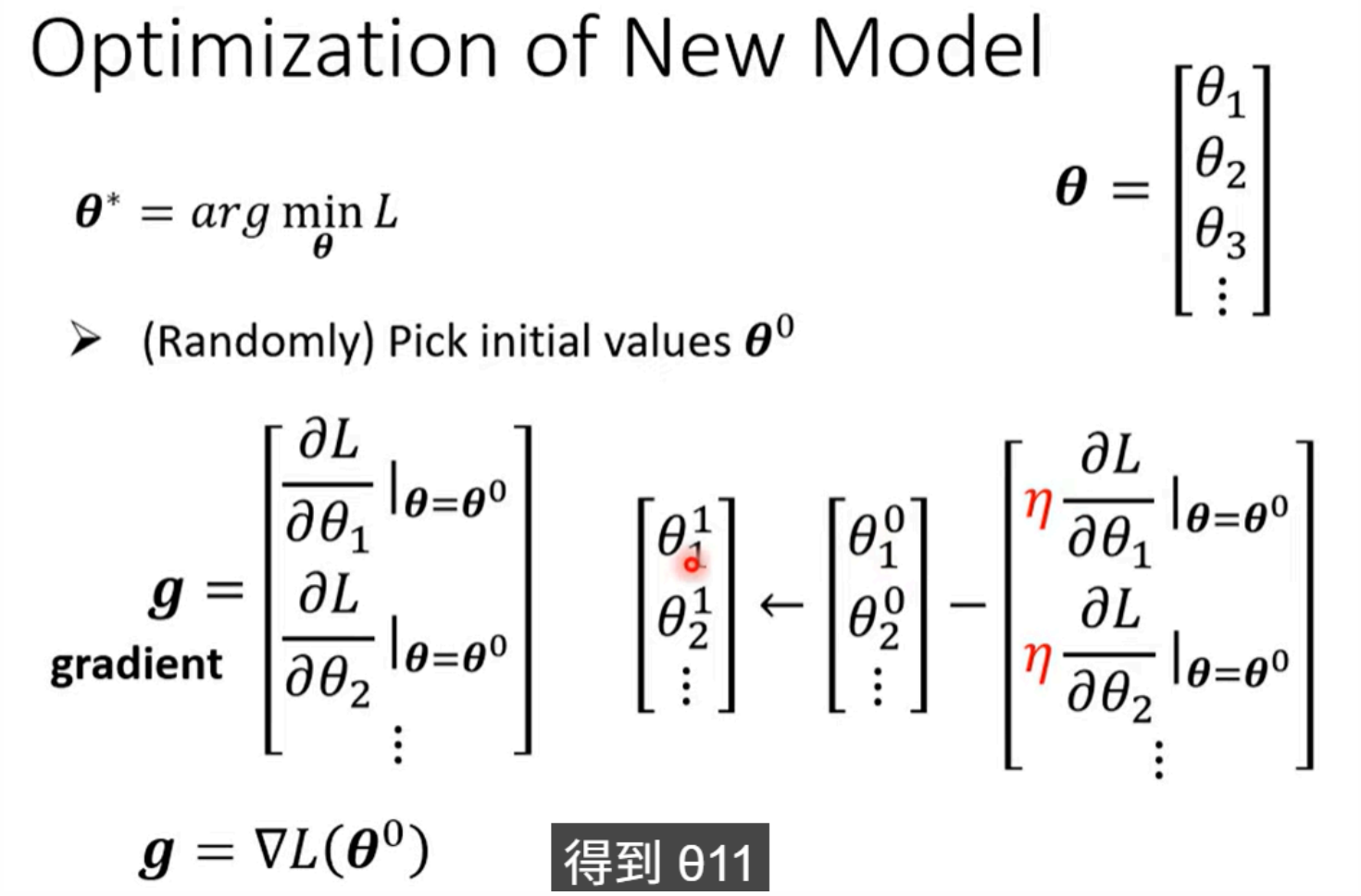


类似于这样我们需要不断对这个参数theta更新来找到函数的最小值使得模型的loss最小或者说最为准确，那么对应的问题就是函数的最小值极小值的判断和选择，数学上我们通过不断求导可以轻易的计算出来函数的最值或者是极小值极大值，对于三位函数我们可以做偏导，但是在train 一个model的时候如果我们要计算三阶导数二阶导数，那么带来的是非常庞大的计算量所我们需要找出一个合适的算法高效并且准确的更新参数。这时候gradient descent就问世了：

2.Vanilla gradient descent或者sgd随机gradient descent



我们随机选取一个初始点然后进行梯度下降每次更新的步长由学习率×当前的梯度，这个梯度的计算之前也讨论过了我们用bp

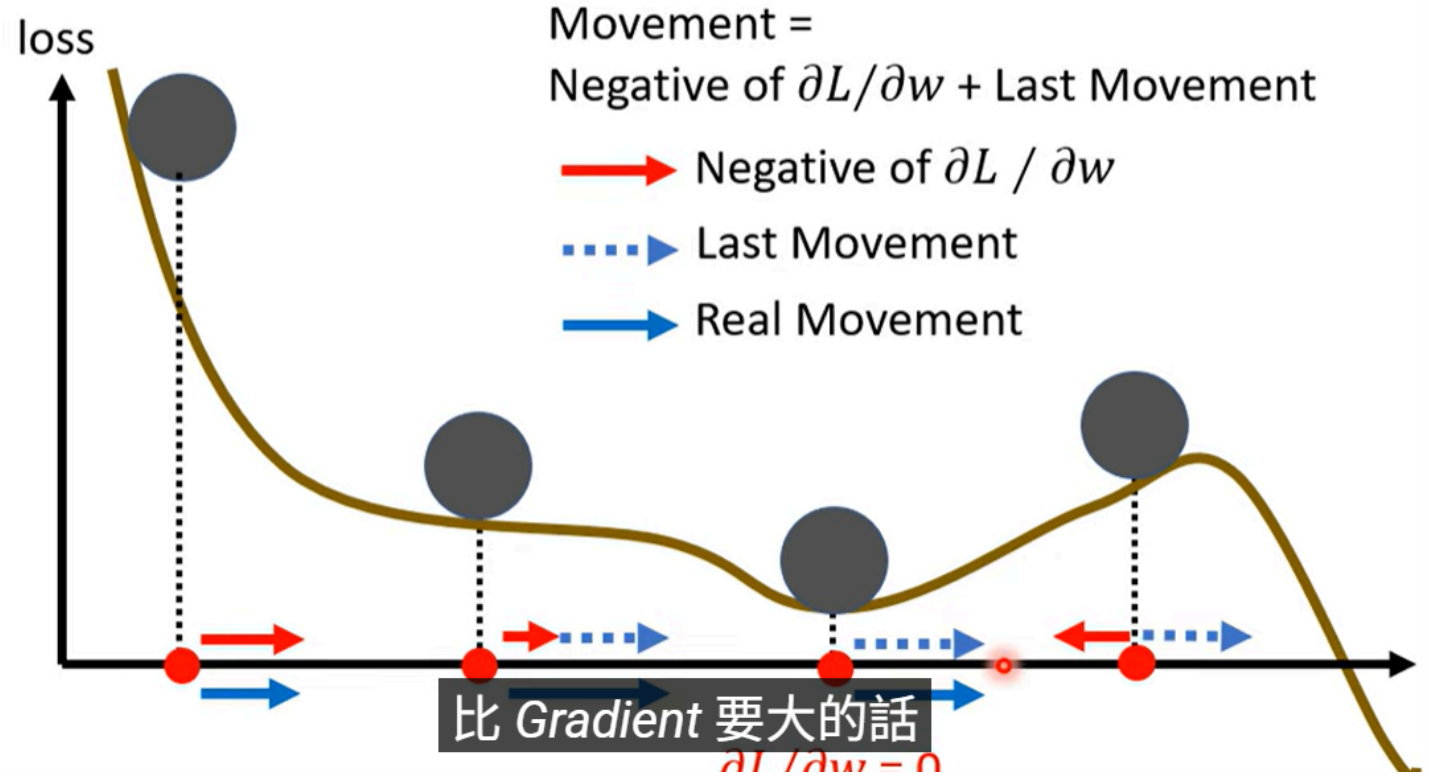


sgd虽然是神但是在很多neuron network里面出现收敛速度极慢而且容易出现在saddle point地方来回打转，就像是下山的时候进入了山谷然后一直在山谷的两侧反复走没办法继续下山，这个在二维函数的时候对应local minima问题然而实际上是个假命题，因为二维世界没有下降的通路不代表三维的函数不能继续gd下降。

3.Momentum：

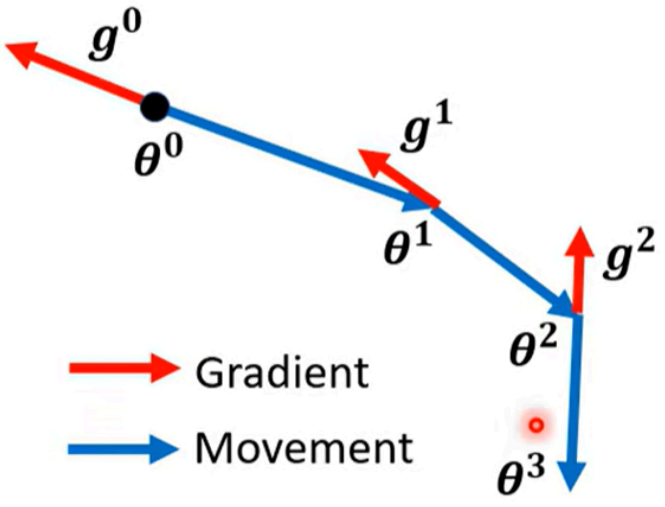
在介绍adam之前我们先介绍一下动量这个物理领域很熟悉的东西。

Gradient Descent + Momentum



想像这个球往下滚的过程，球在谷的位置能继续前进一段可能越过这个坡到达更低点，这个就是动量的表现，和物理世界的动量没有任何区别，那如果我们引入动量进入到下降算法里面是不是就能够解决一定的saddle point的问题呢？那难点就是怎么量化的加入动量到gd中去？

(Vanilla) Gradient Descent



Starting at $\boldsymbol{\theta^0}$

Compute gradient $\boldsymbol{g^0}$

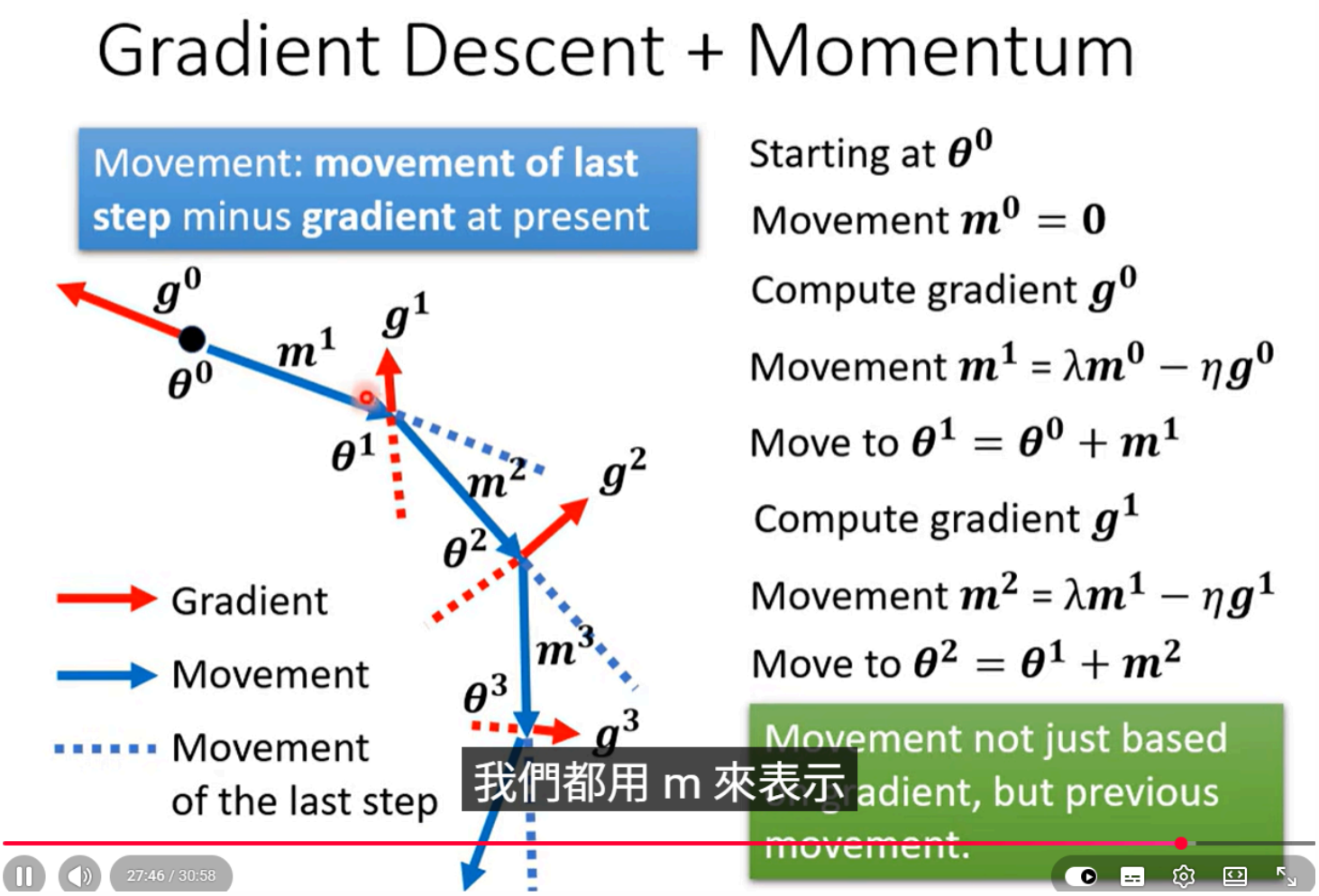
Move to $\boldsymbol{\theta^1} = \boldsymbol{\theta^0} - \eta \boldsymbol{g^0}$

Compute gradient $\boldsymbol{g^1}$

Move to $\boldsymbol{\theta^2} = \boldsymbol{\theta^1} - \eta \boldsymbol{g^1}$

⋮

这个是传统梯度下降的向量表示图



这个是我们的加入动量之后的标识图，可以看到实际上就是做了一个向量的求和，movement变量就是上次的gradient的反方向的延长，梯度是本来没有m的运动方向的反方向。

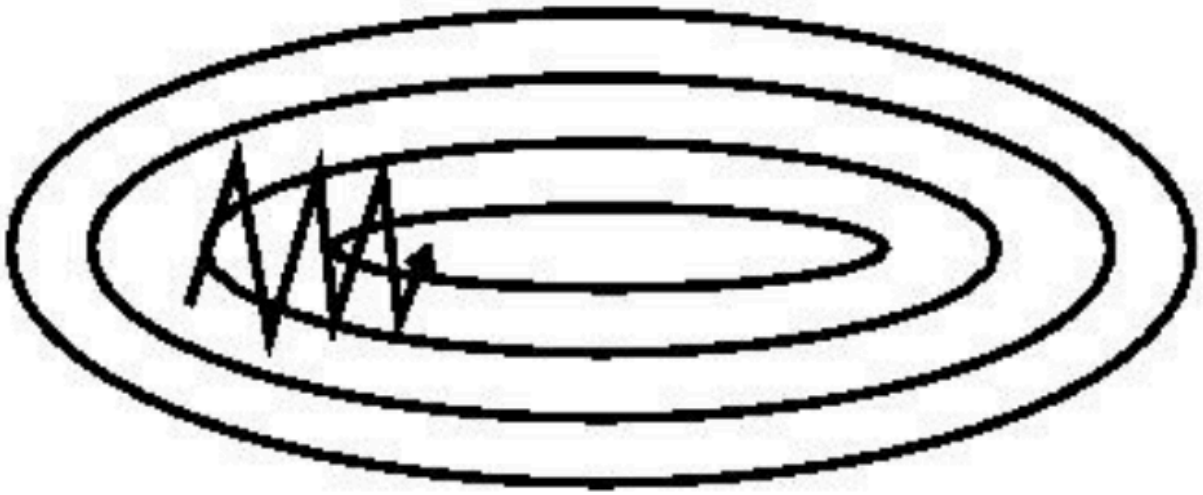


图 1(a): SGD

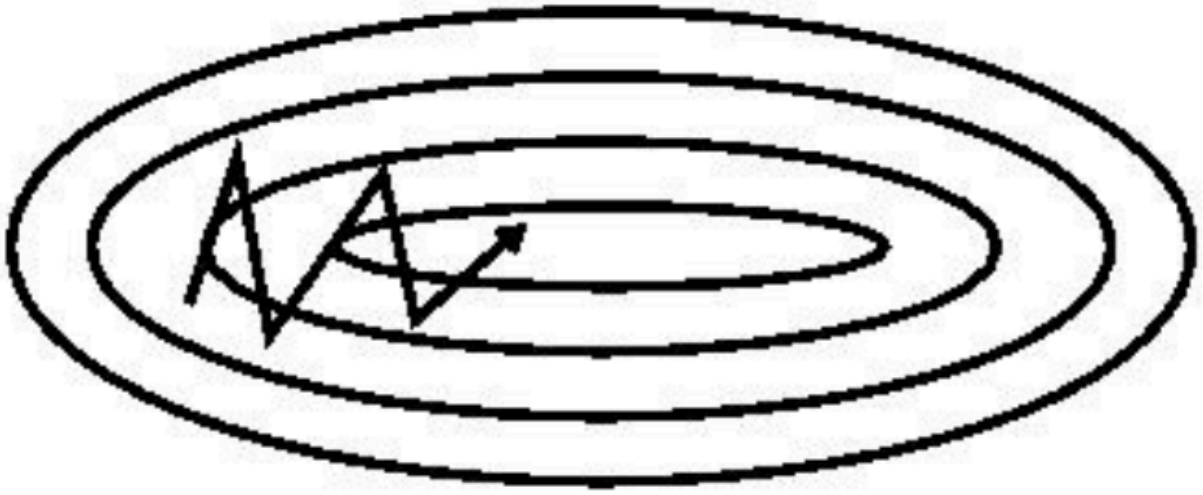


图 1(b): SGD with momentum

可以看到收敛速度变快也更不容易在谷底打转

4.Adam-AdamW：

我们现在遇到新的问题就是，学习率的变化现在作为一个超参数被我们定死了，但是实际上我需要他的变化动态调整，在平缓的地方学习步长变大，陡峭的地方学习步长变小。这时候我们那就得让学习率的变化变成一个动态调整的变量，而adagrad中是直接对伊塔学习率除grad即可，但是只关注单次的梯度缺少全局的视野，于是我们万能的hinton就发明了RMSProp，加入了之前的所有的gi都纳入计算并且给了个权重比例去调整，但是不知道为什么这样子效果就真的会变好，反正实际表现来说是非常好的效果。

RMSProp

$$\theta_i^{t+1} \leftarrow \theta_i^t - \frac{\eta}{\sigma_i^t} g_i^t$$

$$\theta_i^1 \leftarrow \theta_i^0 - \frac{\eta}{\sigma_i^0} g_i^0 \qquad \sigma_i^0 = \sqrt{\left(g_i^0\right)^2}$$

$$0 < \alpha < 1$$

$$\theta_i^2 \leftarrow \theta_i^1 - \frac{\eta}{\sigma_i^1} g_i^1 \qquad \sigma_i^1 = \sqrt{\alpha\left(\sigma_i^0\right)^2 + (1-\alpha)\left(g_i^1\right)^2}$$

$$\theta_i^3 \leftarrow \theta_i^2 - \frac{\eta}{\sigma_i^2} g_i^2 \qquad \sigma_i^2 = \sqrt{\alpha\left(\sigma_i^1\right)^2 + (1-\alpha)\left(g_i^2\right)^2}$$

RMSProp

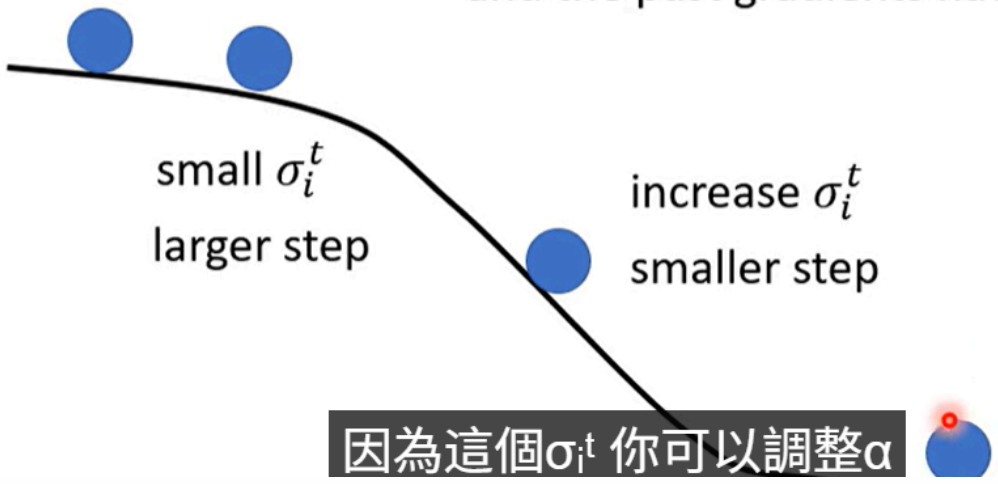
$$\theta_i^{t+1} \leftarrow \theta_i^t - \frac{\eta}{\sigma_i^t} g_i^t$$

$$g_i^1 \; g_i^2 \cdots\cdots g_i^{t-1}$$

$$0 < \alpha < 1$$

$$\sigma_i^t = \sqrt{\alpha\left(\sigma_i^{t-1}\right)^2 + (1-\alpha)\left(g_i^t\right)^2}$$

The recent gradient has larger influence, and the past gradients have less influence



然后当我们把之前介绍的向量momentum纳入参数更新的公式就出现了Adam=RMSProp+Momentum。

Adam: RMSProp + Momentum

Algorithm 1: *Adam*, our proposed algorithm for stochastic optimization. See section 2 for details, and for a slightly more efficient (but less clear) order of computation. g_t^2 indicates the elementwise square $g_t \odot g_t$. Good default settings for the tested machine learning problems are $\alpha = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$ and $\epsilon = 10^{-8}$. All operations on vectors are element-wise. With β_1^t and β_2^t we denote β_1 and β_2 to the power t.

Require: α: Stepsize
Require: $\beta_1, \beta_2 \in [0, 1)$: Exponential decay rates for the moment estimates
Require: $f(\theta)$: Stochastic objective function with parameters θ
Require: θ_0: Initial parameter vector
$m_0 \leftarrow 0$ (Initialize 1st moment vector) → for momentum
$v_0 \leftarrow 0$ (Initialize 2nd moment vector) → for RMSprop
$t \leftarrow 0$ (Initialize timestep)
while θ_t not converged **do**
$t \leftarrow t + 1$
$g_t \leftarrow \nabla_\theta f_t(\theta_{t-1})$ (Get gradients w.r.t. stochastic objective at timestep t)
$m_t \leftarrow \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t$ (Update biased first moment estimate)
$v_t \leftarrow \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$ (Update biased second raw moment estimate)
$\widehat{m}_t \leftarrow m_t / (1 - \beta_1^t)$ (Compute bias-corrected first moment estimate)
$\widehat{v}_t \leftarrow v_t / (1 - \beta_2^t)$ (Compute bias-corrected second raw moment estimate)
$\theta_t \leftarrow \theta_{t-1} - \alpha \cdot \widehat{m}_t$
然後預設的參數不要隨便調
end while

那么sgd既没有adam的momentum又没有动态调整的学习率learning rate scheduling。那么大多时候都是adam表现更加出色。

Algorithm 2 Adam with L_2 regularization and Adam with decoupled weight decay (AdamW)

given $\alpha = 0.001, \beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}, \lambda \in \mathbb{R}$
initialize time step $t \leftarrow 0$, parameter vector $\boldsymbol{\theta}_{t=0} \in \mathbb{R}^n$, first moment vector $\boldsymbol{m}_{t=0} \leftarrow \boldsymbol{0}$, second momen vector $\boldsymbol{v}_{t=0} \leftarrow \boldsymbol{0}$, schedule multiplier $\eta_{t=0} \in \mathbb{R}$
repeat
  $t \leftarrow t + 1$
  $\nabla f_t(\boldsymbol{\theta}_{t-1}) \leftarrow \text{SelectBatch}(\boldsymbol{\theta}_{t-1})$ ▷ select batch and return the corresponding gradient
  $\boldsymbol{g}_t \leftarrow \nabla f_t(\boldsymbol{\theta}_{t-1}) + \lambda \boldsymbol{\theta}_{t-1}$
  $\boldsymbol{m}_t \leftarrow \beta_1 \boldsymbol{m}_{t-1} + (1 - \beta_1) \boldsymbol{g}_t$ ▷ here and below all operations are element-wise
  $\boldsymbol{v}_t \leftarrow \beta_2 \boldsymbol{v}_{t-1} + (1 - \beta_2) \boldsymbol{g}_t^2$
  $\hat{\boldsymbol{m}}_t \leftarrow \boldsymbol{m}_t / (1 - \beta_1^t)$ ▷ β_1 is taken to the power of t
  $\hat{\boldsymbol{v}}_t \leftarrow \boldsymbol{v}_t / (1 - \beta_2^t)$ ▷ β_2 is taken to the power of t
  $\eta_t \leftarrow \text{SetScheduleMultiplier}(t)$ ▷ can be fixed, decay, or also be used for warm restarts
  $\boldsymbol{\theta}_t \leftarrow \boldsymbol{\theta}_{t-1} - \eta_t \left(\alpha \hat{\boldsymbol{m}}_t / (\sqrt{\hat{\boldsymbol{v}}_t} + \epsilon) + \lambda \boldsymbol{\theta}_{t-1} \right)$
until *stopping criterion is met*
return optimized parameters $\boldsymbol{\theta}_t$

知乎 @barely

接下来登场的就是weight decay 在adam基础上引入权重衰减，就是直接加入到loss中去

loss = loss + weight decay parameter * L2 norm of the weights。

不细致的描述了有时候也会用这个。

VII.总结

要想用 Pytorch 训练任何神经网络，训练回圈包含 5 个基础步骤：

1. `output = model(input)` 是模型的前向传播，接收输入并生成输出。
2. `J = loss(output, target <or> label)` 接收模型的输出，并计算训练时对于真实目标的损失。
3. `model.zero_grad()` 清除计算的梯度，使它们不会积累到下次的传播。
4. `J.backward()` 进行反向传播和累加：它为每个我们指定了 `requires_grad=True` 的变数 `x` 计算 $\nabla_t extttx J$。这些会累计到每个变数的梯度：$\mathbf{x.grad} \leftarrow \mathbf{x.grad} + \nabla_x J$。
5. `optimiser.step()` 进行梯度下降的一步：$\vartheta \leftarrow \vartheta - \eta \nabla_\vartheta J$。

当训练一个神经网络，很有可能你要依序进行这五个步骤。

五步思考也是封装抽象模块化的思路。

学一点理论就不会对黑盒这样恐惧了。